

Lecture 12:

Optimization Techniques



Markus Hohle

University of California, Berkeley

**Numerical Methods for
Computational Science**

Course Map

Week 1:	Introduction to Scientific Computing and Python Libraries
Week 2:	Linear Algebra Fundamentals
Week 3:	Vector Calculus
Week 4:	Numerical Differentiation and Integration
Week 5:	Solving Nonlinear Equations
Week 6:	Probability Theory Basics
Week 7:	Random Variables and Distributions
Week 8:	Statistics for Data Science
Week 9:	Eigenvalues and Eigenvectors
Week 10:	Simulation and Monte Carlo Method
Week 11:	Data Fitting and Regression
Week 12:	Optimization Techniques
Week 13:	Machine Learning Fundamentals



Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



source: SKRyanrr

Outline

Lagrangian Multiplier

Gradient Descent Again

- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning



Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



source: SKRyanrr

Outline

Lagrangian Multiplier

Gradient Descent Again

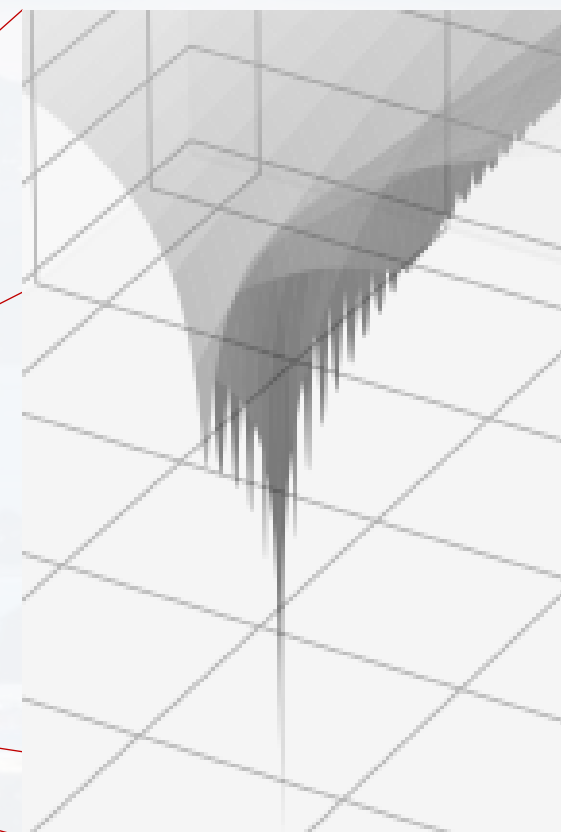
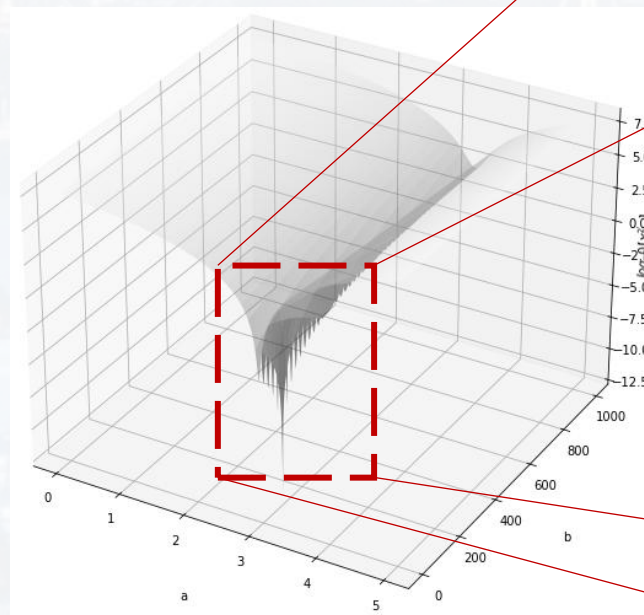
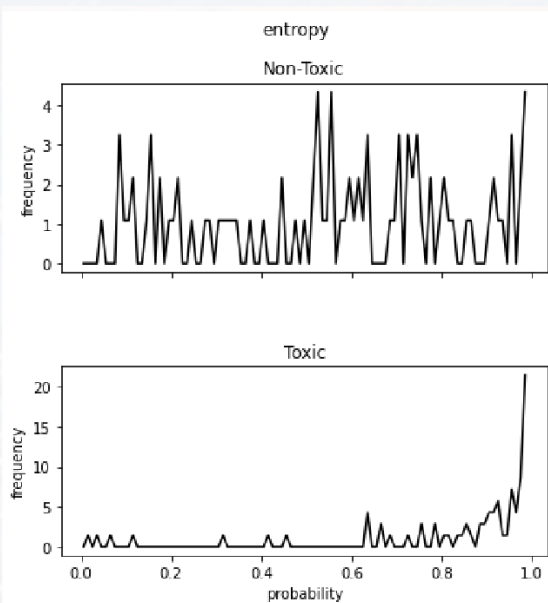
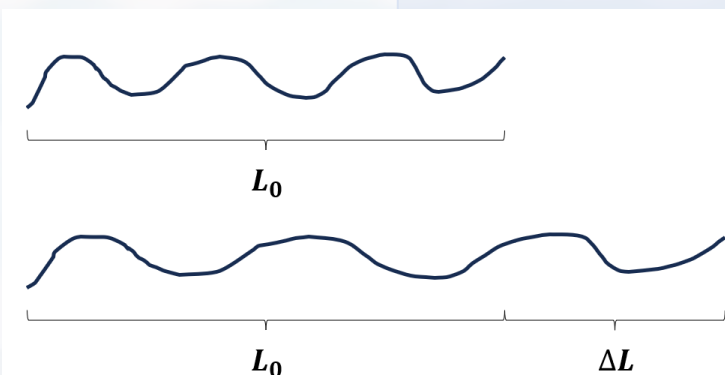
- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning



problem: often we need to find an **extreme** of a function:

- equilibrium at **minimum** energy
- distributions for **maximum** entropy
- regression: **minimizing** χ^2_{red} or MSE
- classification: **minimizing** cross-entropy etc

$$P(k|c) = \frac{(c \Delta t)^k e^{-c \Delta t}}{k!}$$

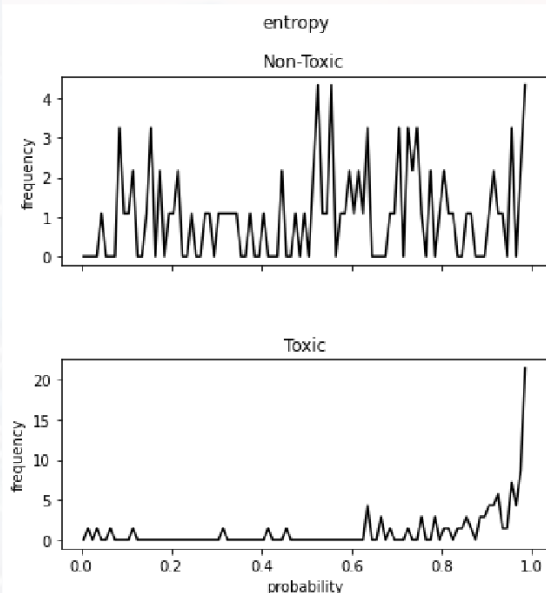
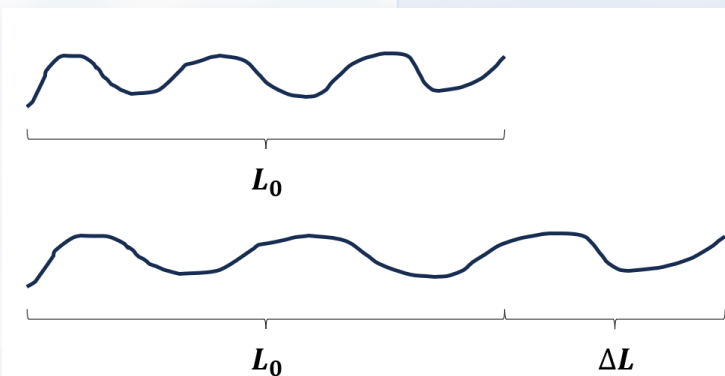




problem: often we need to find an **extreme** of a function:

- equilibrium at **minimum** energy
- distributions for **maximum** entropy
- regression: **minimizing** χ^2_{red} or MSE
- classification: **minimizing** cross-entropy etc

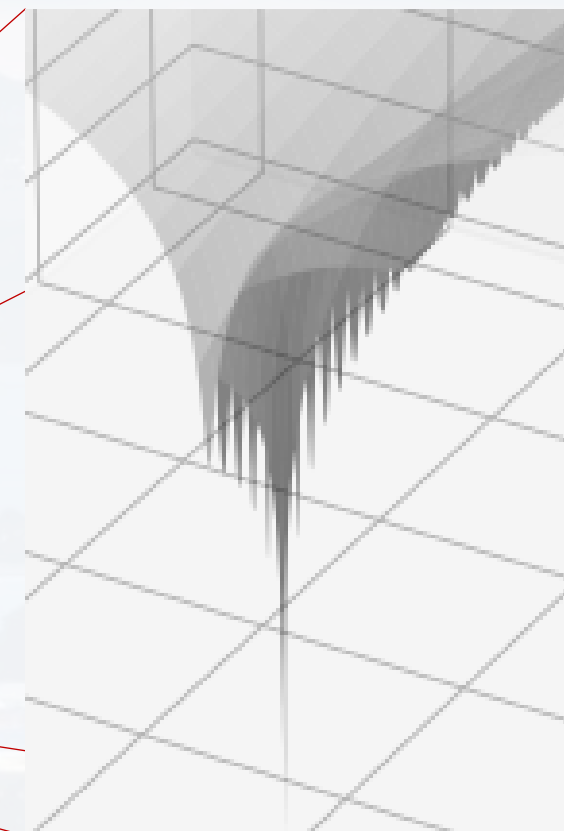
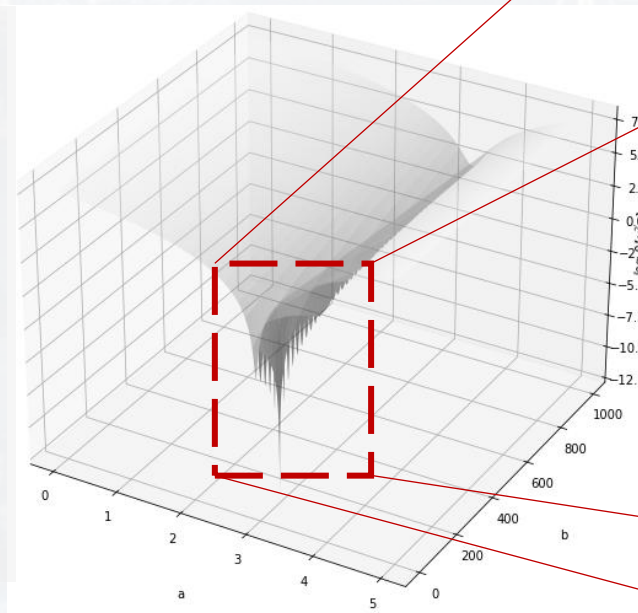
$$P(k|c) = \frac{(c \Delta t)^k e^{-c \Delta t}}{k!}$$

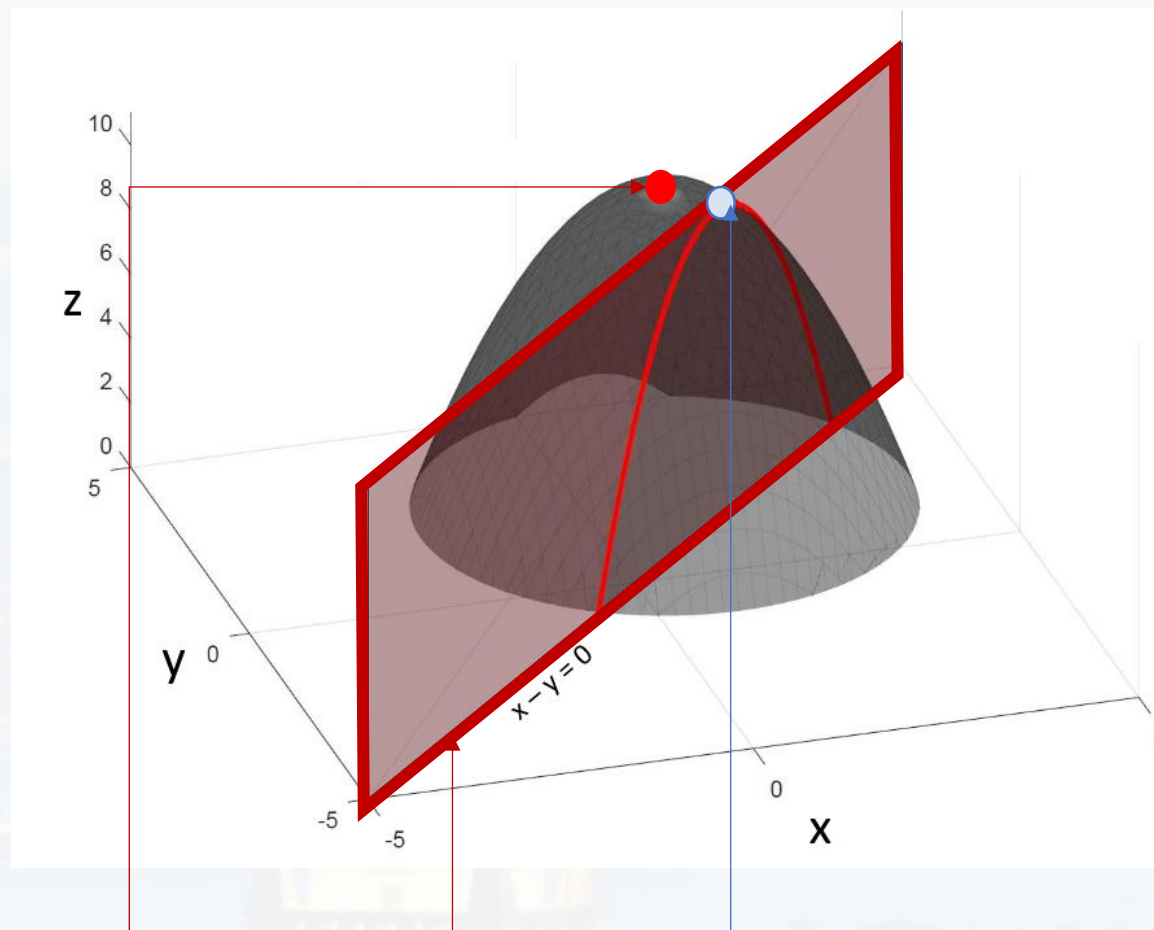


but often, systems are **subject to a set of constrains**

- mass can only be positive
- energy is constant
- probabilities sum up to 1
- ...

goal: find **extreme within these constrains!**





maximum of the function

$$f(x, y) = z = -(x - 2)^2 - (y - 1)^2 + 10$$

$$\frac{\partial f(x, y)}{\partial x} = 0 \quad \text{and} \quad \frac{\partial f(x, y)}{\partial y} = 0$$

$$-2(x - 2) = 0$$

$$-2(y - 1) = 0$$

$$x = 2$$

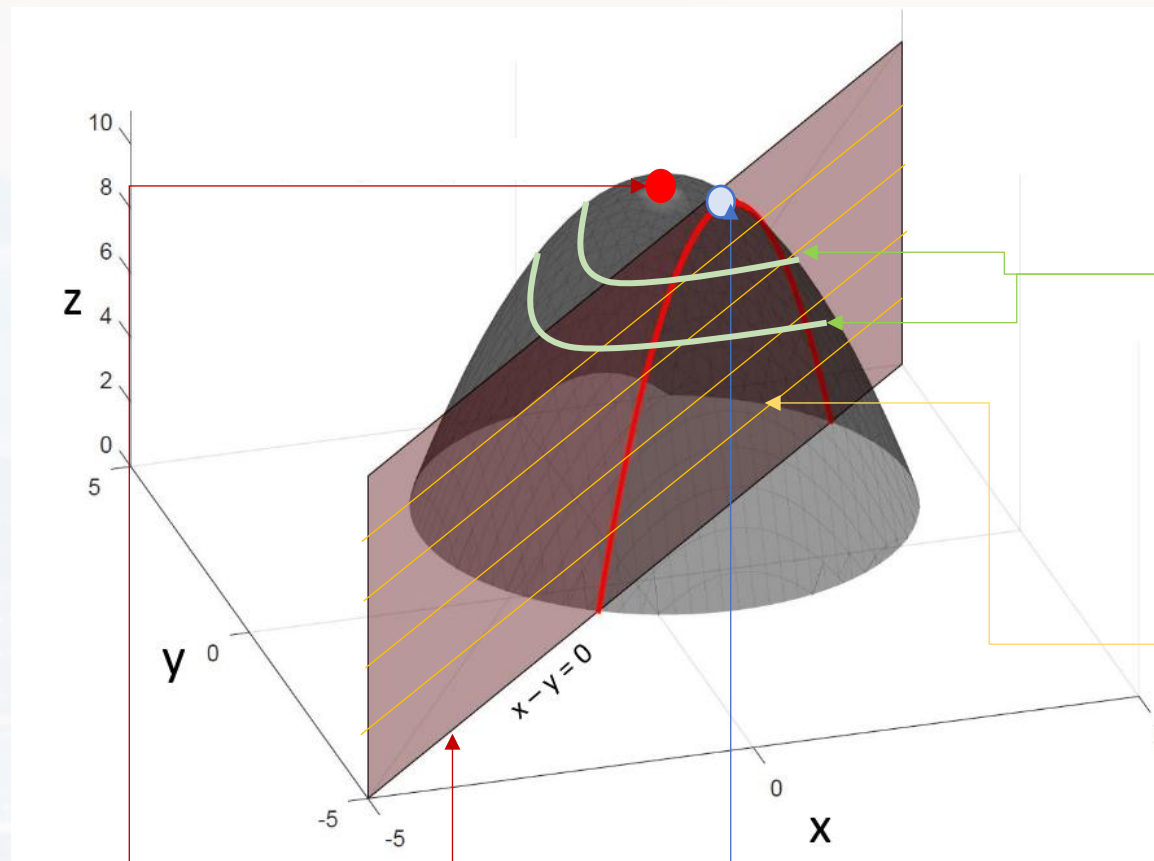
$$y = 1$$

$$f(2, 1) = 10$$

$$\begin{aligned} z &= 10 \text{ at} \\ x &= 2 \\ y &= 1 \end{aligned}$$

$$\text{constrain } g(x, y) = x - y = 0$$

$$\text{maximum of the function, subject to } g(x, y)$$



maximum of the function

$$f(x, y) = z = -(x - 2)^2 - (y - 1)^2 + 10$$

level lines $f(x, y) = \text{const}$

$$\begin{aligned} df(x, y) &= \frac{\partial f(x, y)}{\partial x} dx + \frac{\partial f(x, y)}{\partial y} dy = 0 \\ &= \text{grad} f \, d\vec{r} = 0 \end{aligned}$$

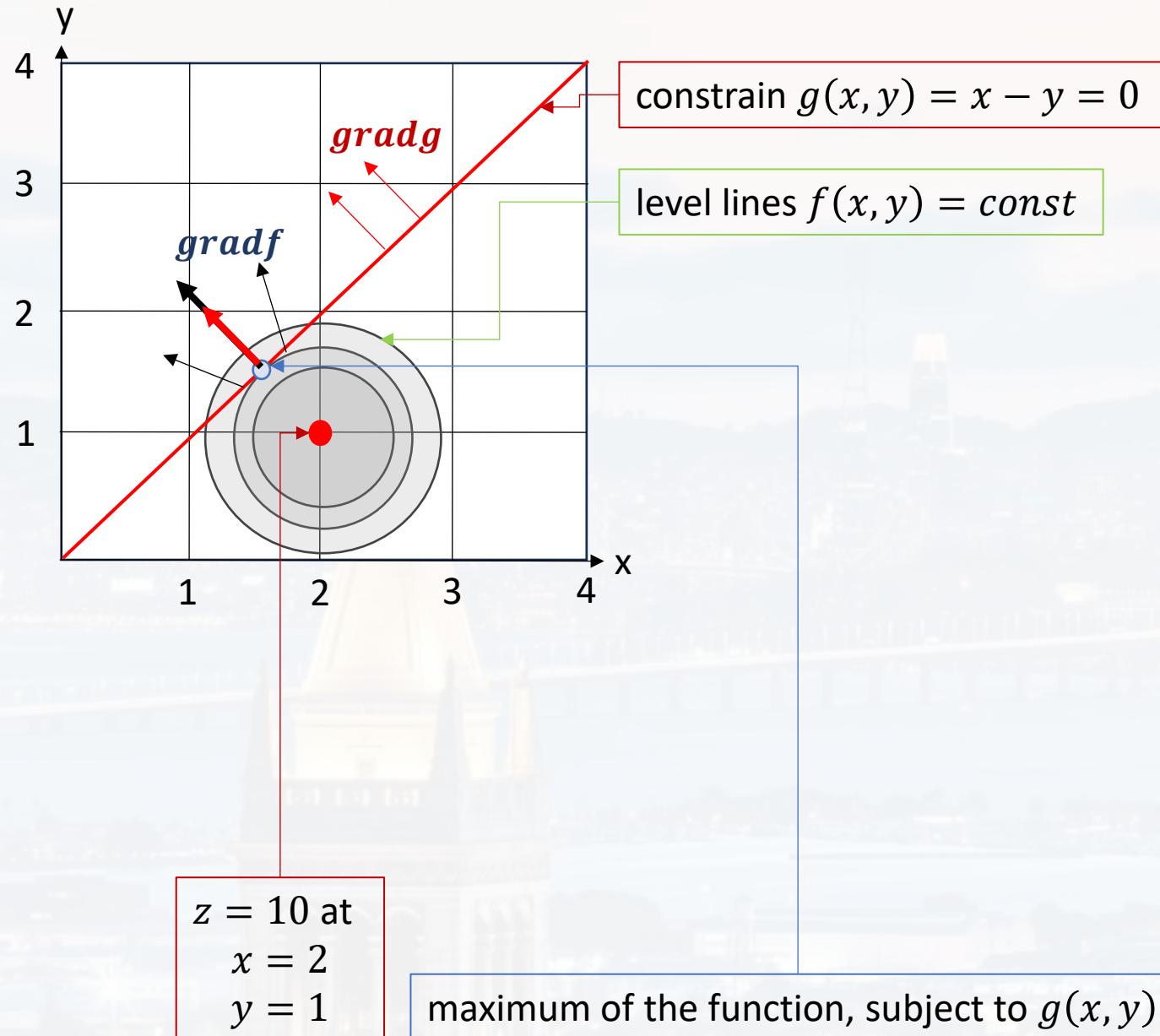
level lines $g(x, y) = \text{const}$

$$\begin{aligned} dg(x, y) &= \frac{\partial g(x, y)}{\partial x} dx + \frac{\partial g(x, y)}{\partial y} dy = 0 \\ &= \text{grad} g \, d\vec{r} = 0 \end{aligned}$$

$z = 10$ at
 $x = 2$
 $y = 1$

constrain $g(x, y) = x - y = 0$

maximum of the function, subject to $g(x, y)$



the maximum of $f(x, y)$
subject to $g(x, y)$ located
where:

$$df(x, y) = dg(x, y)$$

$$\text{grad} f \, d\vec{r} = \text{grad} g \, d\vec{r}$$

$$\text{grad} f = \text{grad} g$$

Both gradients need to point
in the same direction
(hence, can be multiplied with a constant,
say λ)!

$$\text{grad} f = \lambda \text{grad} g$$

λ **Lagrangian Multiplier**



the maximum of $f(x, y)$ subject to $g(x, y)$

$$\text{grad} f = \lambda \text{grad} g$$

$$\text{grad} f - \lambda \text{grad} g = 0$$

$$\int \text{grad} f \, d\vec{r} - \lambda \int \text{grad} g \, d\vec{r} = \text{const}$$

$$f(x, y) - \lambda g(x, y) = \text{const}$$

the Lagrangian
 $L(x, y, \lambda)$

more general:

$$L(x_1, x_2, \dots, x_i, x_N, \lambda_1, \lambda_2, \dots, \lambda_k, \lambda_K) = f(x_1, x_2, \dots, x_i, x_N) - \sum_{k=1}^K \lambda_k g_k(x_1, x_2, \dots, x_i, x_N)$$

N dimensions and $K \leq N$ constraints



the maximum of $f(x, y)$ subject to $g(x, y)$

$$\text{grad} f = \lambda \text{grad} g$$

$$\text{grad} f - \lambda \text{grad} g = 0$$

$$\int \text{grad} f \, d\vec{r} - \lambda \int \text{grad} g \, d\vec{r} = \text{const}$$

$$f(x, y) - \lambda g(x, y) = \text{const}$$

the Lagrangian
 $L(x, y, \lambda)$

more general:

physics:

describes dynamics of a system based on conserved quantities (energy, momentum etc) $\rightarrow L = \text{const}$

optimization:

more robust results (see later)

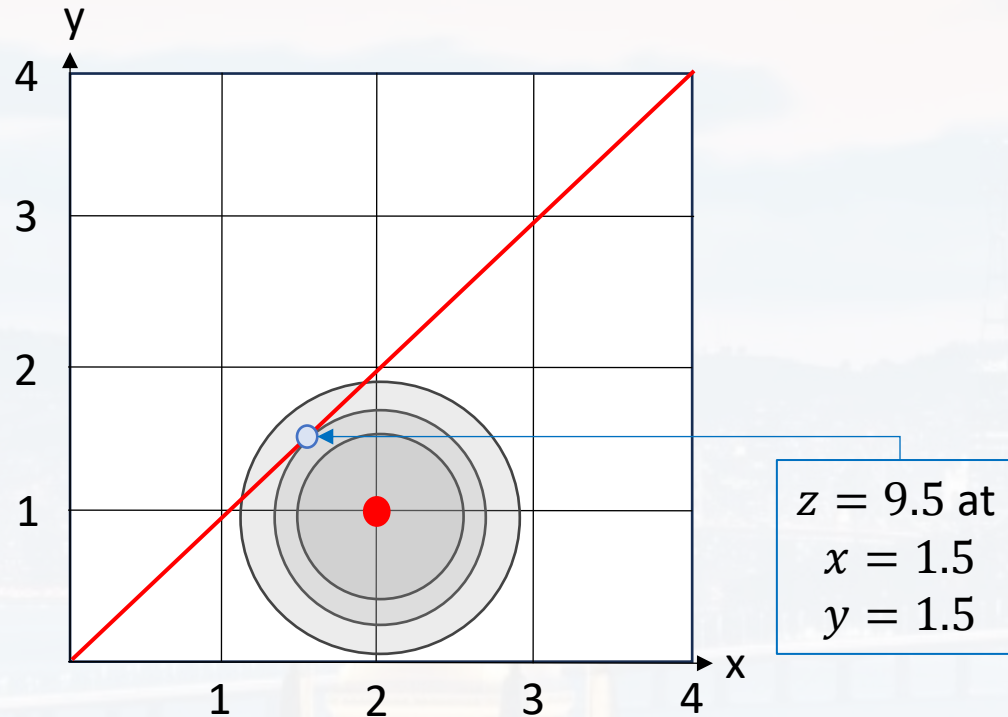
machine learning: loss function

$$L(x_1, x_2, \dots, x_i, x_N, \lambda_1, \lambda_2, \dots, \lambda_k, \lambda_K) = f(x_1, x_2, \dots, x_i, x_N) - \sum_{k=1}^K \lambda_k g_k(x_1, x_2, \dots, x_i, x_N)$$

N dimensions and $K \leq N$ constraints



the maximum of $f(x, y)$ subject to $g(x, y)$



maximum of the function

$$f(x, y) = z = -(x - 2)^2 - (y - 1)^2 + 10$$

$$\text{constrain } g(x, y) = x - y = 0$$

$$\text{constrain } x = y \quad \begin{aligned} x &= 1.5 \\ y &= 1.5 \end{aligned}$$

$$f(1.5, 1.5) = 9.5$$

$$\text{grad} f = \lambda \text{grad} g \quad \frac{\partial f(x, y)}{\partial x} = \lambda \frac{\partial g(x, y)}{\partial x}$$

$$\frac{\partial f(x, y)}{\partial y} = \lambda \frac{\partial g(x, y)}{\partial y}$$

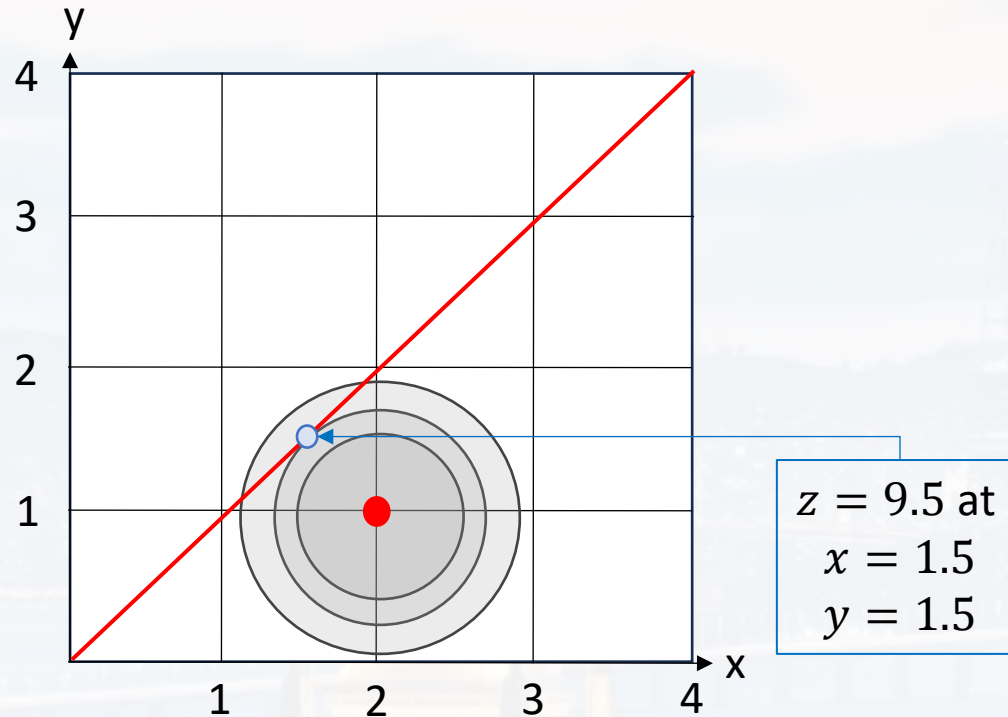
$$-2(x - 2) = \lambda$$

$$-2(y - 1) = -\lambda$$

$$y = -x + 3$$



the maximum of $f(x, y)$ subject to $g(x, y)$



maximum of the function

$$f(x, y) = z = -(x - 2)^2 - (y - 1)^2 + 10$$

$$\text{constrain } g(x, y) = x - y = 0$$

$$\begin{aligned} \text{constrain } x &= y & x &= 1.5 \\ & & y &= 1.5 \end{aligned}$$

$$f(1.5, 1.5) = 9.5$$

We need to solve $N + K$ equations!

$$L(x_1, x_2, \dots, x_i, x_N, \lambda_1, \lambda_2, \dots, \lambda_k, \lambda_K) = f(x_1, x_2, \dots, x_i, x_N) - \sum_{k=1}^K \lambda_k g_k(x_1, x_2, \dots, x_i, x_N)$$

N dimensions and $K \leq N$ constraints



maximum entropy of flipping a coin:

$$f(p_1, p_2) = -p_1 \ln p_1 - p_2 \ln p_2$$

subject to

$$g(p_1, p_2) = p_1 + p_2 = 1$$

absolute maximum:

$$\frac{\partial f(p_1, p_2)}{\partial p_1} = 0$$

$$\frac{\partial f(p_1, p_2)}{\partial p_2} = 0$$

$$-\ln p_1 - 1 = 0$$

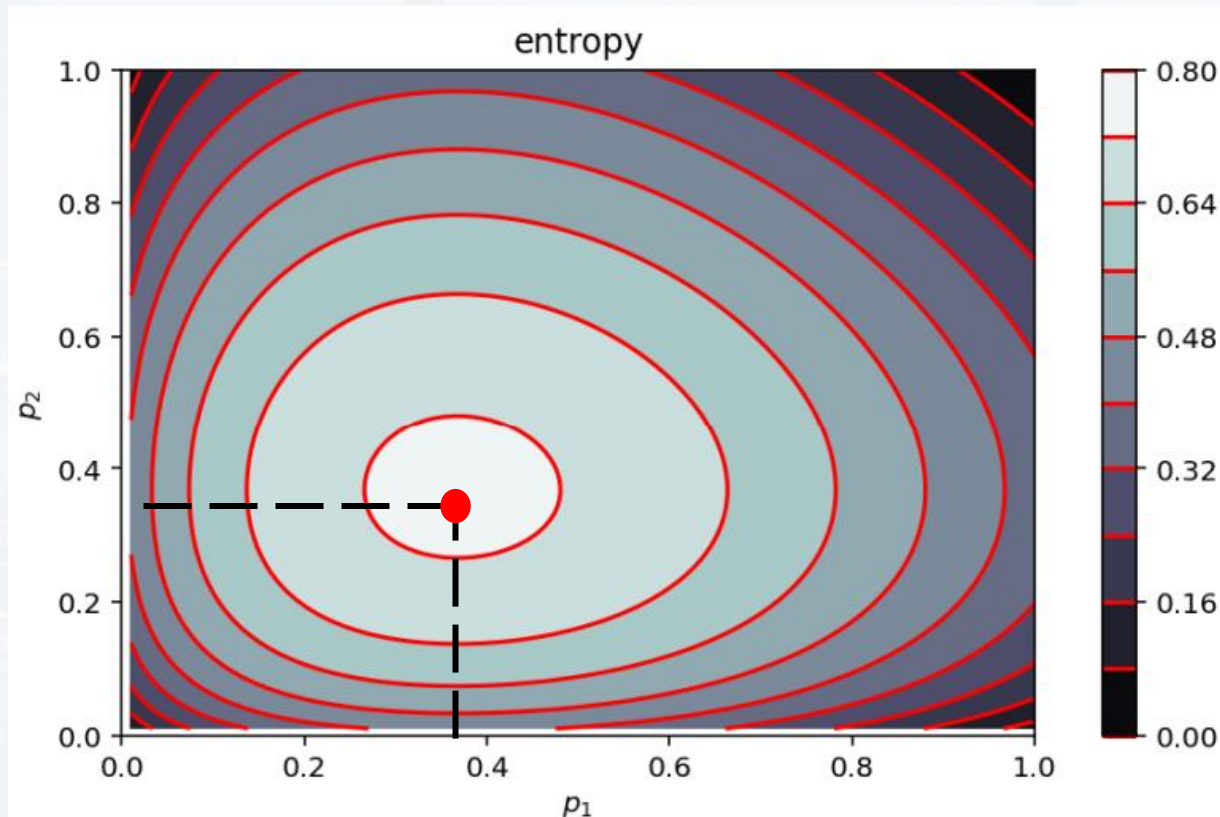
$$-\ln p_2 - 1 = 0$$

$$p_1 = p_2 = \frac{1}{e}$$

$$f\left(\frac{1}{e}, \frac{1}{e}\right) = \frac{2}{e} \approx 0.74$$

however:

$$p_1 + p_2 = \frac{2}{e} \neq 1$$





maximum entropy of flipping a coin:

$$f(p_1, p_2) = -p_1 \ln p_1 - p_2 \ln p_2$$

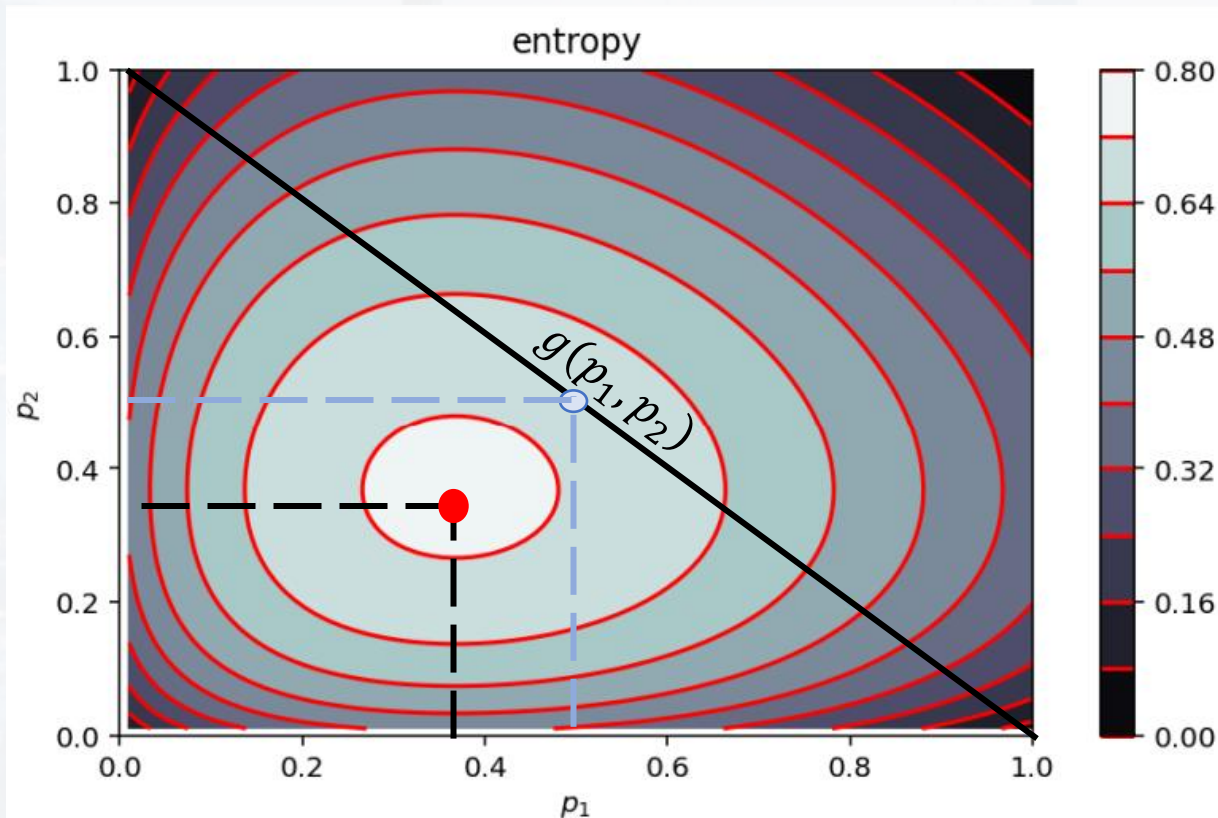
subject to

$$g(p_1, p_2) = p_1 + p_2 = 1$$

maximum subject to $g(p_1, p_2)$:

$$\frac{\partial f(p_1, p_2)}{\partial p_1} = \lambda \frac{\partial g(p_1, p_2)}{\partial p_1}$$

$$\frac{\partial f(p_1, p_2)}{\partial p_2} = \lambda \frac{\partial g(p_1, p_2)}{\partial p_2}$$



$$-\ln p_1 - 1 = \lambda$$

$$-\ln p_2 - 1 = \lambda$$

$$p_1 = p_2$$

$$\text{constrain: } p_1 + p_2 = 1$$

$$p_1 = p_2 = \frac{1}{2}$$

$$f\left(\frac{1}{2}, \frac{1}{2}\right) = \ln 2 \approx 0.69$$

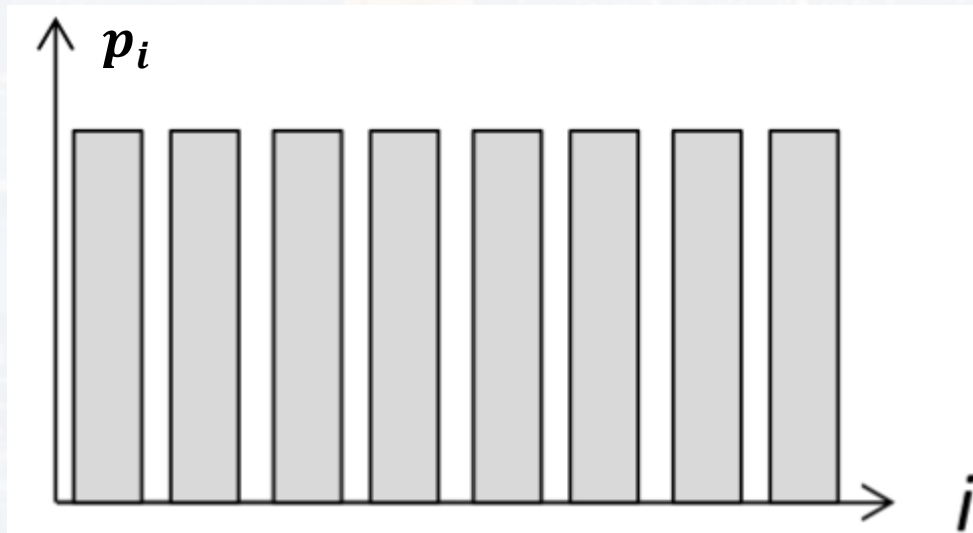


maximum entropy for I states:

$$f(p_1, \dots, p_i, \dots, p_I) = - \sum_{i=1}^I p_i \ln p_i$$

subject to

$$g(p_1, \dots, p_i, \dots, p_I) = \sum_{i=1}^I p_i = 1$$



maximum subject to $g(p_1, \dots, p_i, \dots, p_I)$:

$$\frac{\partial f(p_1, \dots, p_i, \dots, p_I)}{\partial p_i} = \lambda \frac{\partial g(p_1, \dots, p_i, \dots, p_I)}{\partial p_i}$$

$$-\ln p_i - 1 = \lambda$$

$$p_i = e^{-(1+\lambda)}$$

constrain:

$$\sum_{i=1}^I e^{-(1+\lambda)} = 1$$

$$I e^{-(1+\lambda)} = 1$$

$$e^{-(1+\lambda)} = \frac{1}{I}$$

probabilities are constant!
→ flat distribution!

$$p_i = \frac{1}{I}$$



Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



source: SKRyanrr

Outline

Lagrangian Multiplier

Gradient Descent Again

- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning

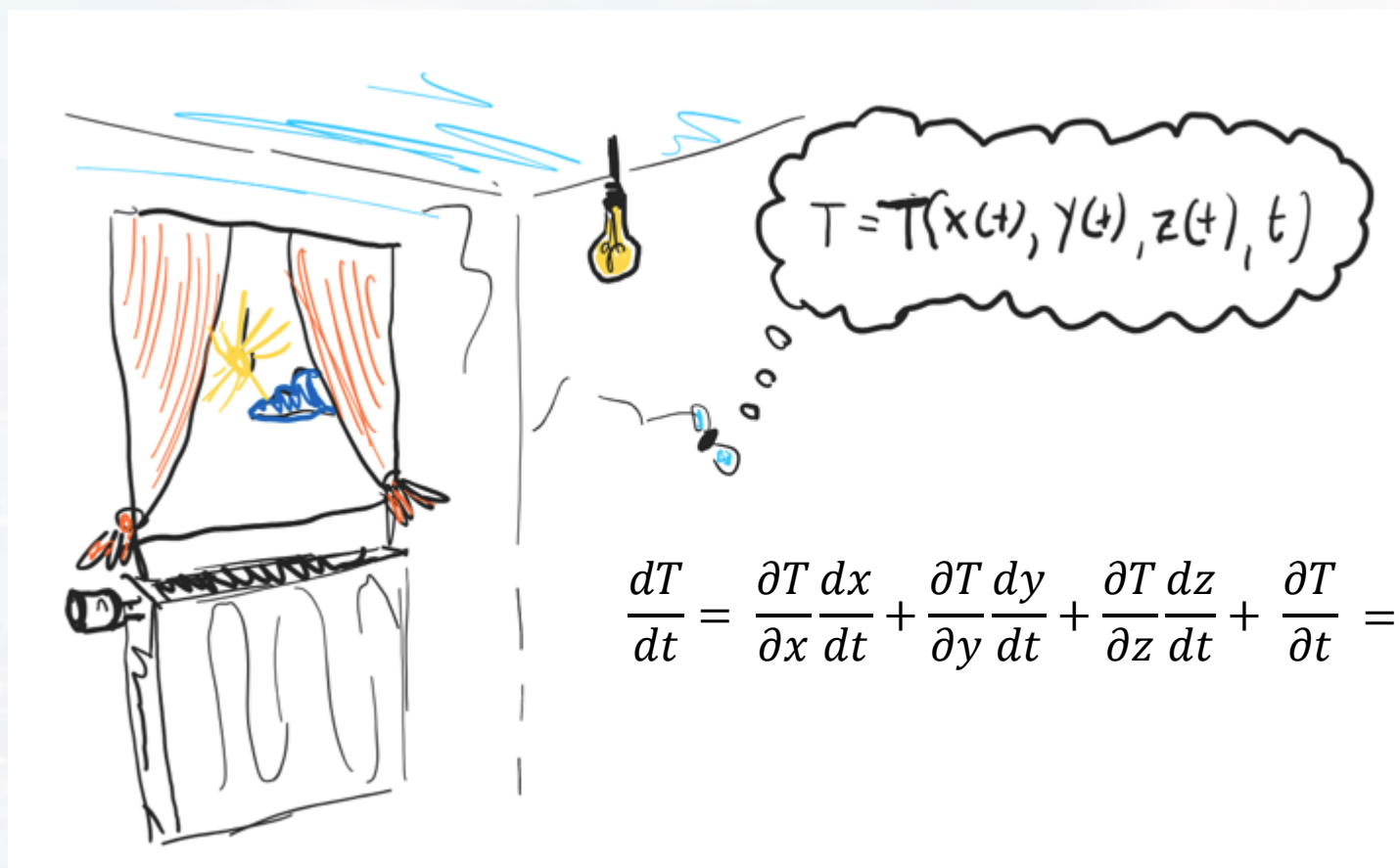


Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

→ extreme of an objective function

gradient
descent

temperature profile in space and time



T : temperature

$$\begin{aligned} \frac{dT}{dt} &= \frac{\partial T}{\partial x} \frac{dx}{dt} + \frac{\partial T}{\partial y} \frac{dy}{dt} + \frac{\partial T}{\partial z} \frac{dz}{dt} + \frac{\partial T}{\partial t} = \begin{pmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \\ \frac{\partial T}{\partial z} \end{pmatrix} \circ \begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \\ \frac{dz}{dt} \end{pmatrix} + \frac{\partial T}{\partial t} \\ &= \text{grad}(T) \circ \vec{v} + \frac{\partial T}{\partial t} \end{aligned}$$



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

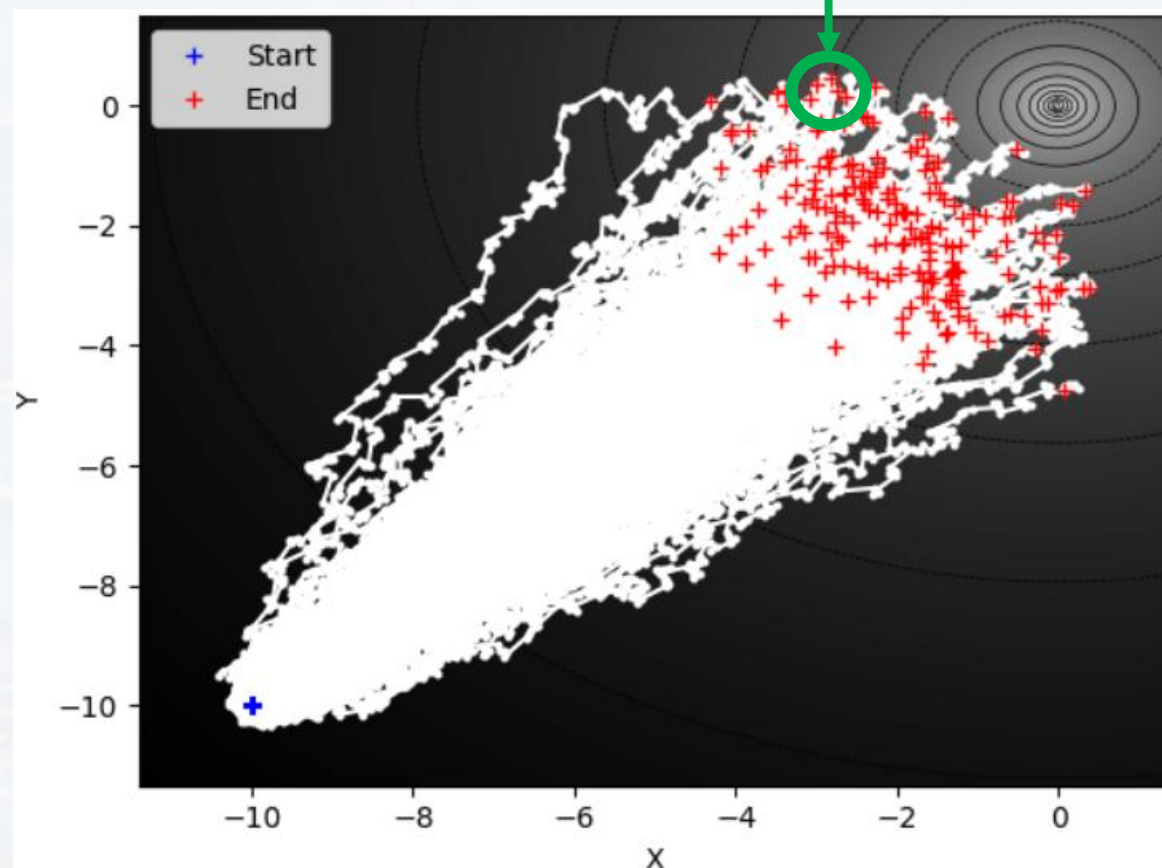
→ extreme of an objective function

gradient
descent

concentration profile in space and time

E. Coli

c: concentration



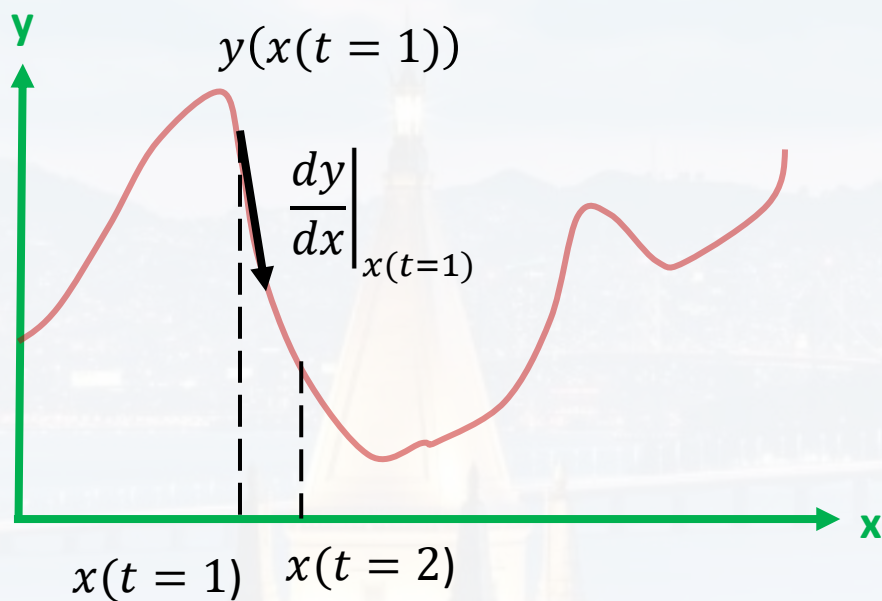
$$\frac{dc}{dt} = \frac{\partial c}{\partial x} \frac{dx}{dt} + \frac{\partial c}{\partial y} \frac{dy}{dt} + \frac{\partial c}{\partial z} \frac{dz}{dt} + \frac{\partial c}{\partial t}$$

$$= \begin{pmatrix} \frac{\partial c}{\partial x} \\ \frac{\partial c}{\partial y} \\ \frac{\partial c}{\partial z} \end{pmatrix} \circ \begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \\ \frac{dz}{dt} \end{pmatrix} + \frac{\partial c}{\partial t}$$

$$= \text{grad}(c) \circ \vec{v} + \frac{\partial c}{\partial t}$$

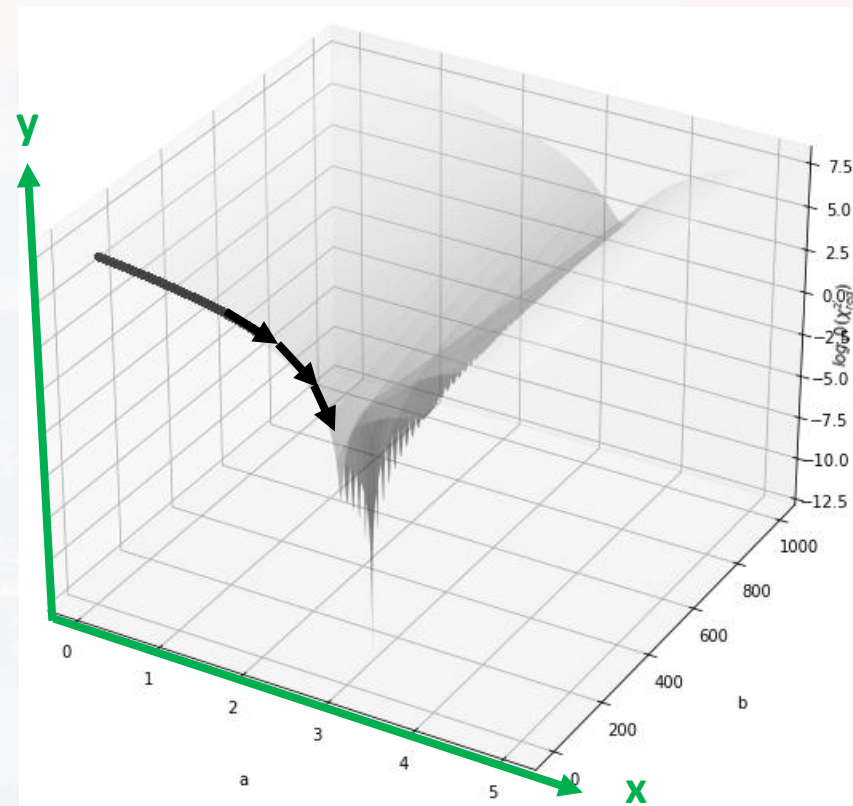


$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



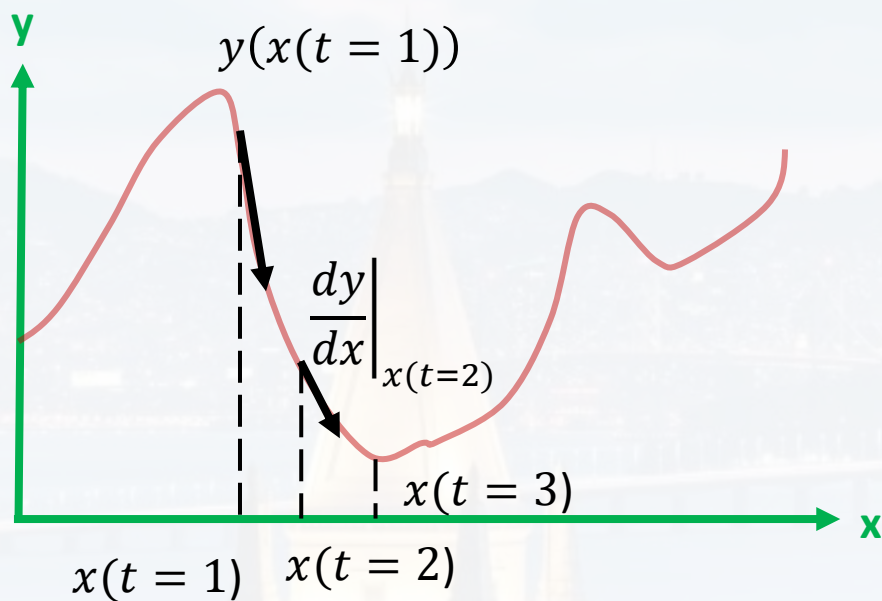
$$x(t=2) = x(t=1) - \varepsilon \left. \frac{dy}{dx} \right|_{x(t=1)}$$

$$\varepsilon > 0$$



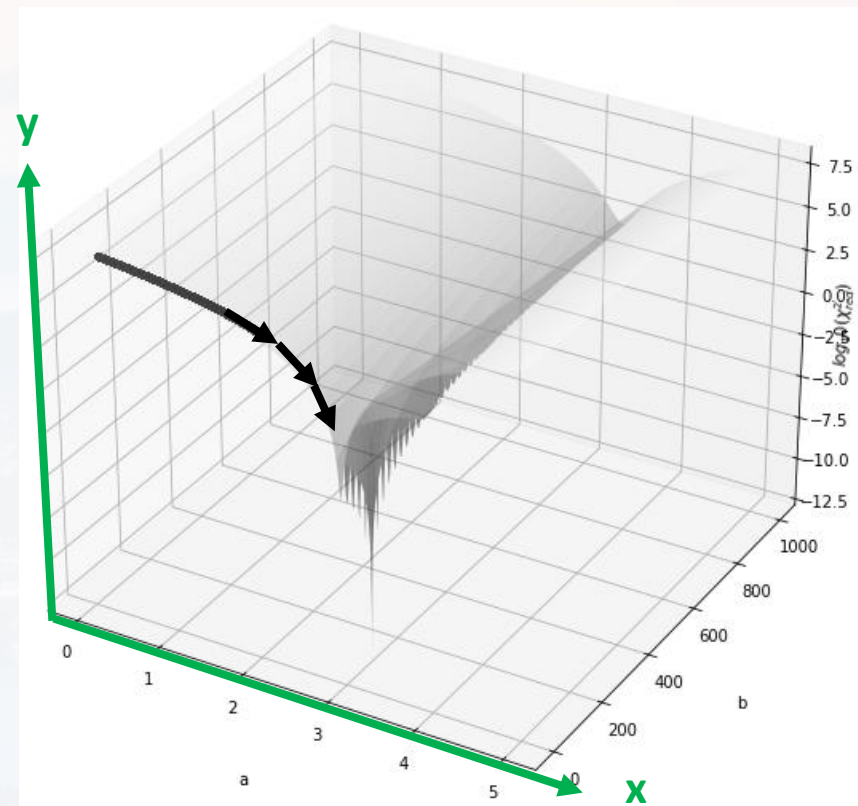


$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



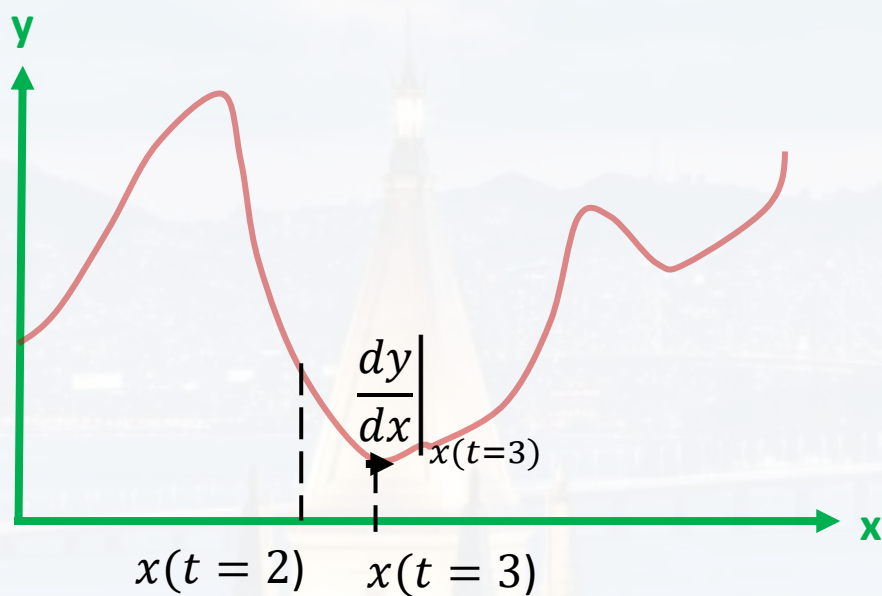
$$x(t=3) = x(t=2) - \epsilon \left. \frac{dy}{dx} \right|_{x(t=2)}$$

$$\epsilon > 0$$



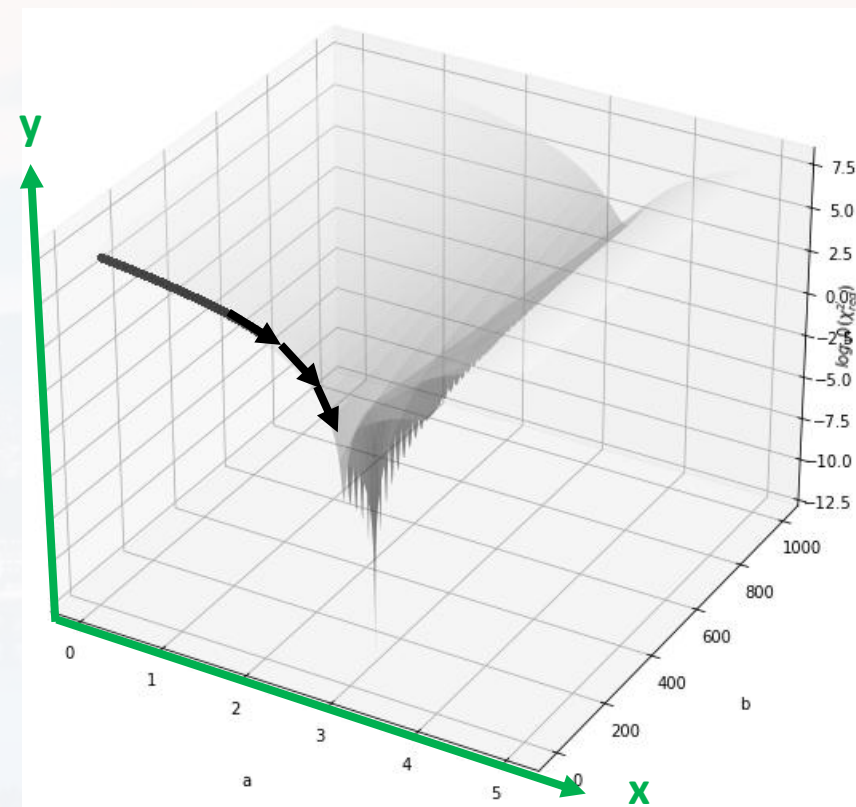


$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



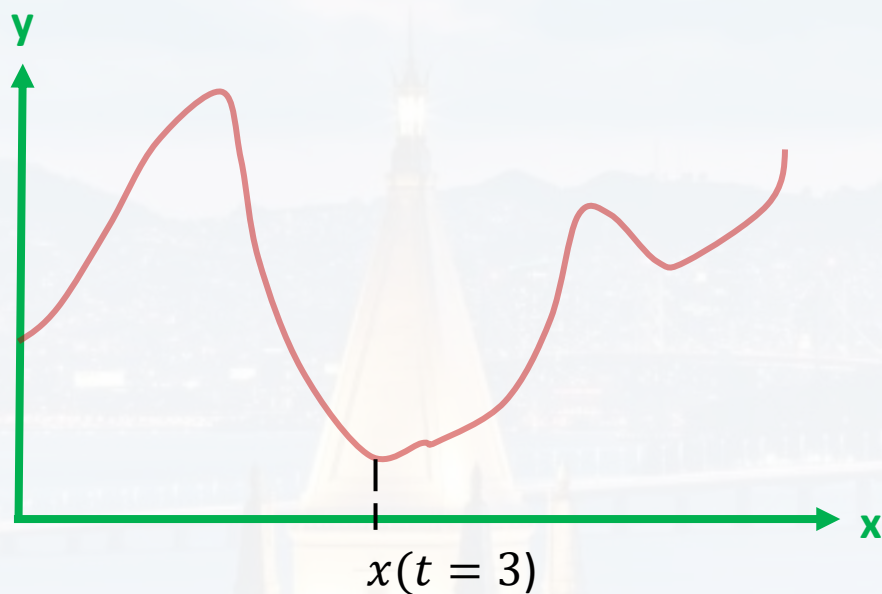
$$x(t=4) = x(t=3) - \varepsilon \left. \frac{dy}{dx} \right|_{x(t=3)}$$

$$\varepsilon > 0$$



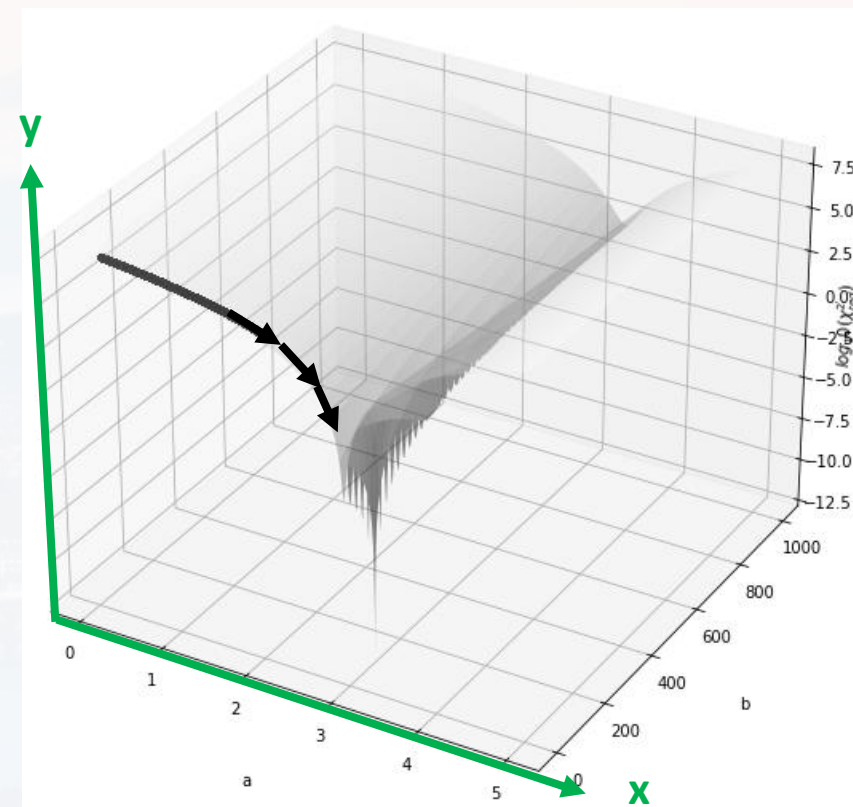


$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



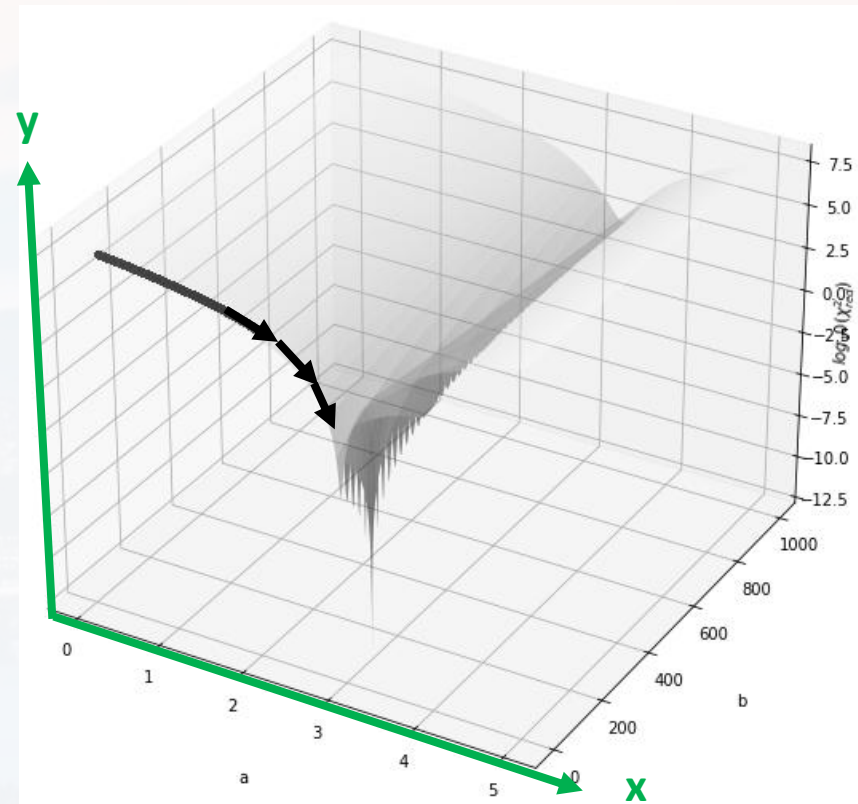
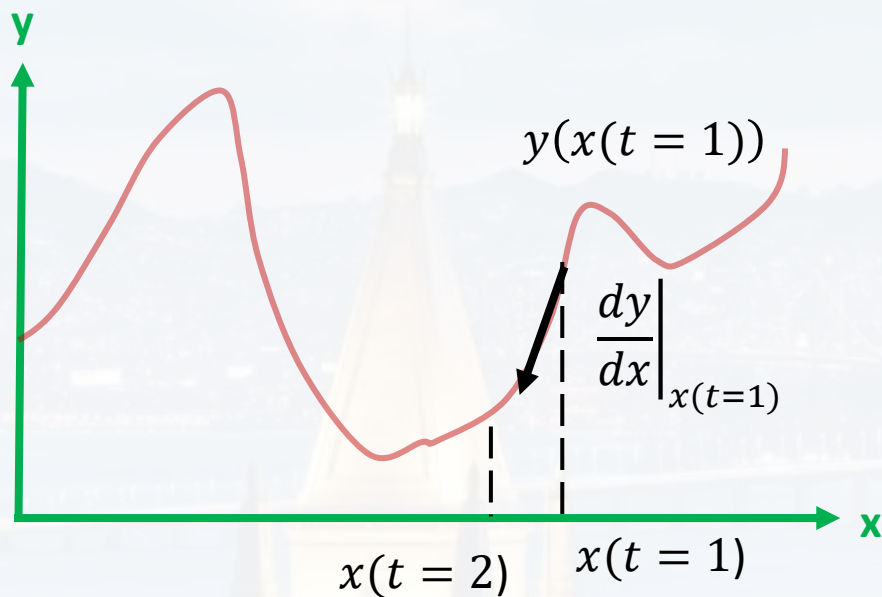
$$x(t=4) = x(t=3) - \varepsilon \left. \frac{dy}{dx} \right|_{x(t=3)}$$

$\varepsilon > 0$





$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



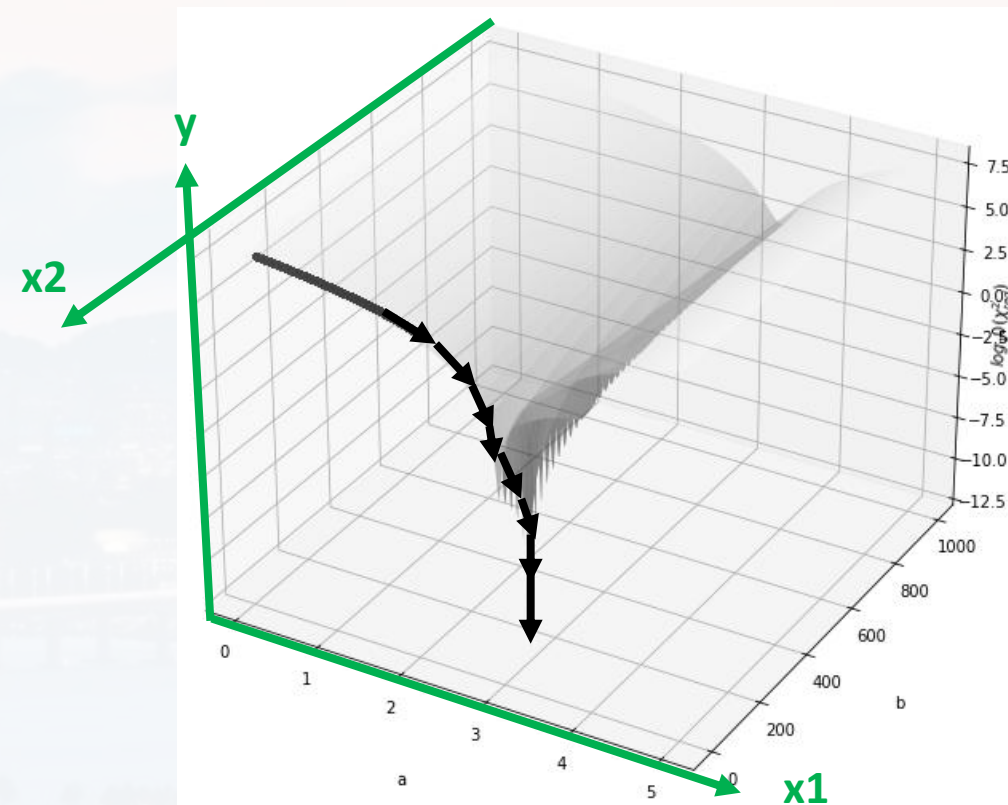
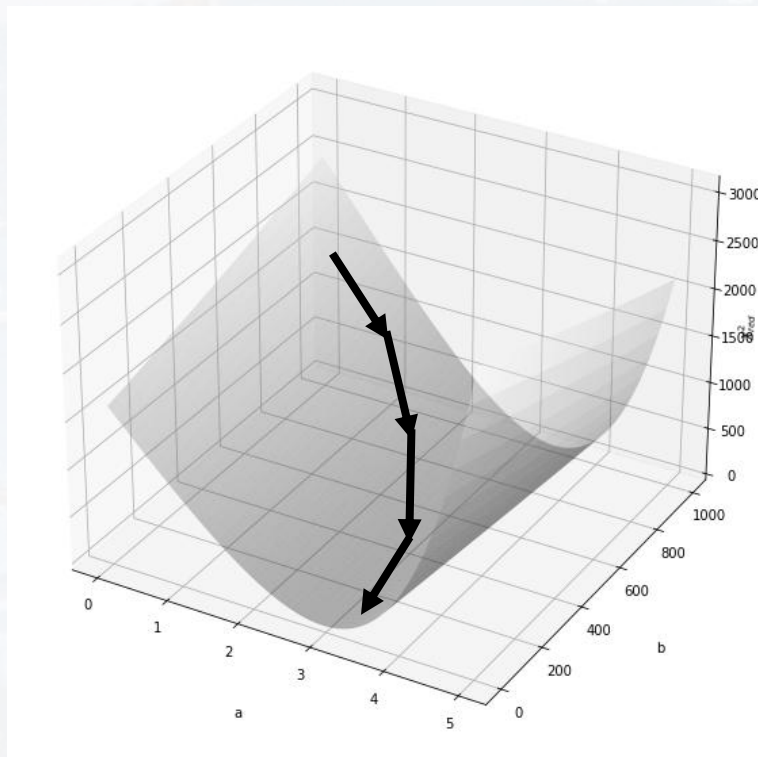
$$x(t=2) = x(t=1) - \epsilon \left. \frac{dy}{dx} \right|_{x(t=1)}$$

$$\epsilon > 0$$



$$\left. \frac{\partial y}{\partial x_1} \right|_{x_1^*; x_2^*} \approx \frac{y(x_1^* + \Delta x_1, x_2^*) - y(x_1^* - \Delta x_1, x_2^*)}{2\Delta x_1}$$

$$\left. \frac{\partial y}{\partial x_2} \right|_{x_1^*; x_2^*} \approx \frac{y(x_1^*, x_2^* + \Delta x_2) - y(x_1^*, x_2^* - \Delta x_2)}{2\Delta x_2}$$





$$\left. \frac{\partial y}{\partial x_1} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(x_1^* + \Delta x_1, x_2^*, \dots, x_N^*) - y(x_1^* - \Delta x_1, x_2^*, \dots, x_N^*)}{2\Delta x_1}$$

$$\left. \frac{\partial y}{\partial x_2} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(x_1^*, x_2^* + \Delta x_2, \dots, x_N^*) - y(x_1^*, x_2^* - \Delta x_2, \dots, x_N^*)}{2\Delta x_2}$$

⋮

$$\left. \frac{\partial y}{\partial x_i} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(\dots, x_i^* + \Delta x_i, \dots, x_N^*) - y(\dots, x_i^* - \Delta x_i, \dots, x_N^*)}{2\Delta x_i}$$

⋮

$$\left. \frac{\partial y}{\partial x_N} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(x_1^*, x_2^*, \dots, x_N^* + \Delta x_N) - y(x_1^*, x_2^*, \dots, x_N^* - \Delta x_N)}{2\Delta x_N}$$

$$\begin{pmatrix} \left. \frac{\partial y}{\partial x_1} \right|_{x_1^*; x_2^*; \dots; x_N^*} \\ \dots \\ \left. \frac{\partial y}{\partial x_i} \right|_{x_1^*; x_2^*; \dots; x_N^*} \\ \dots \\ \left. \frac{\partial y}{\partial x_N} \right|_{x_1^*; x_2^*; \dots; x_N^*} \end{pmatrix}$$

$= \text{grad}(y)_x$
gradient of y wrt x



$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

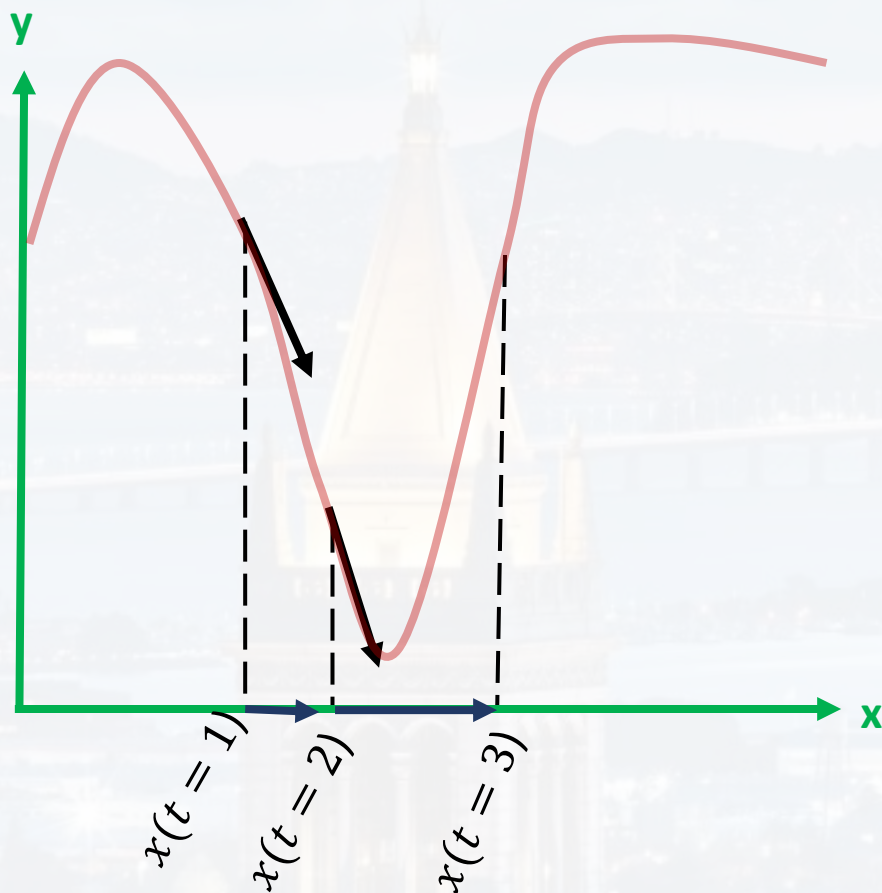
$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

called *learning rate*

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is





Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



source: SKRyanrr

Outline

Lagrangian Multiplier

Gradient Descent Again

- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning



$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

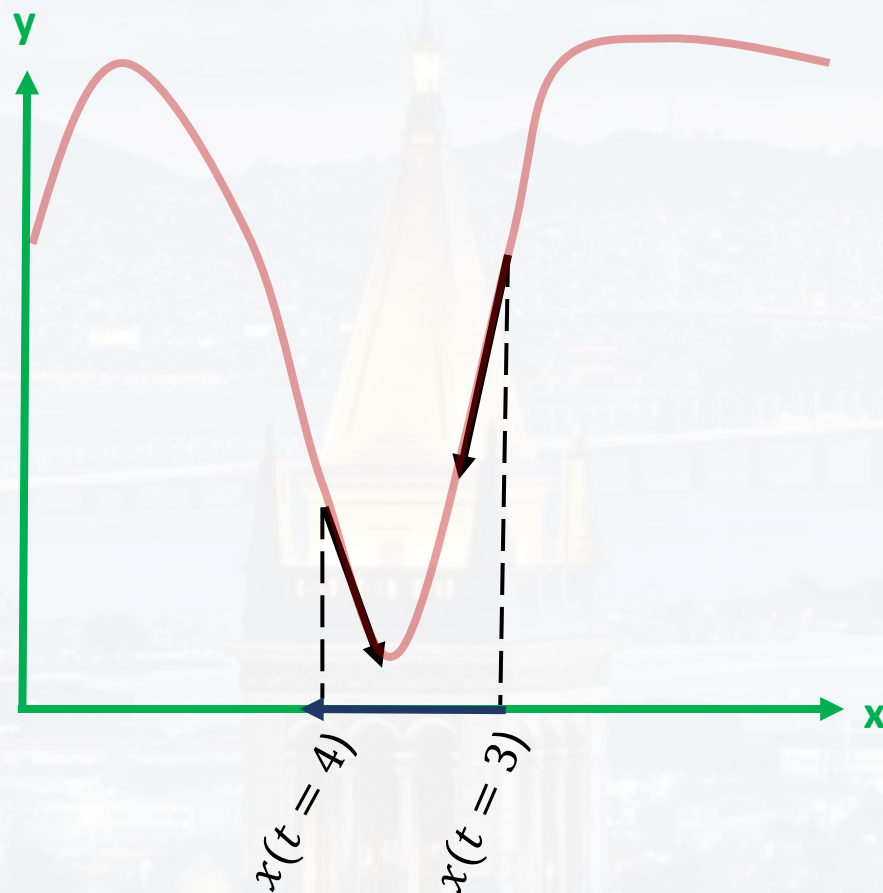
$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

called *learning rate*

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is





$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

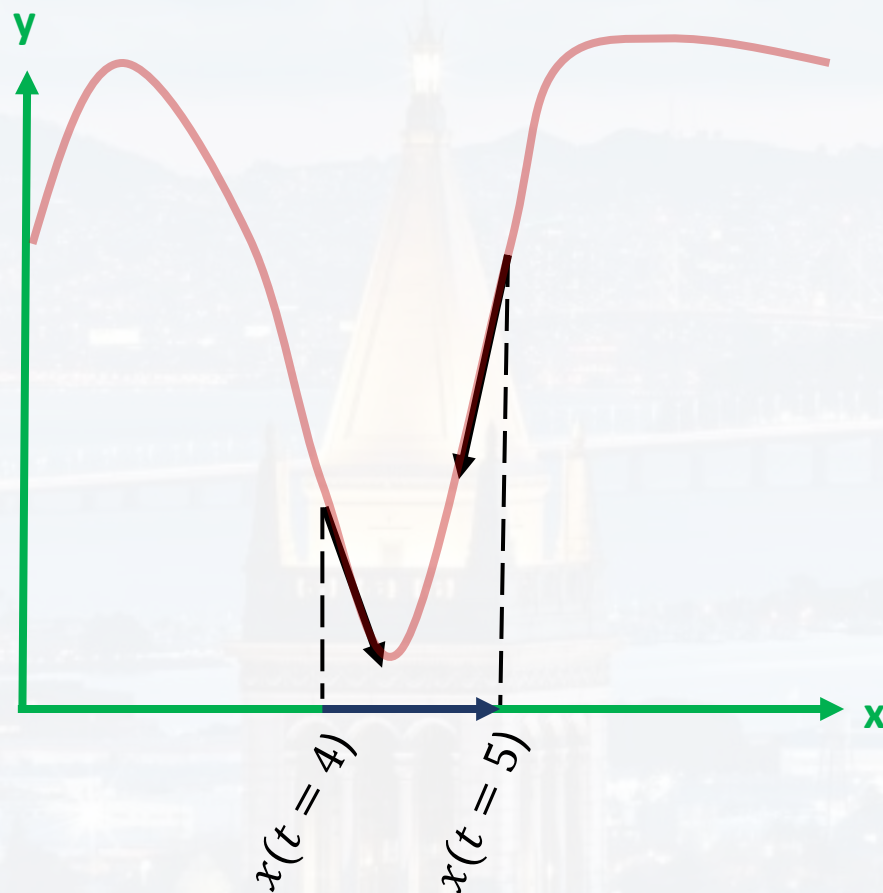
called *learning rate*

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is

... and so on...

→ smaller ϵ ?





$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

called *learning rate*

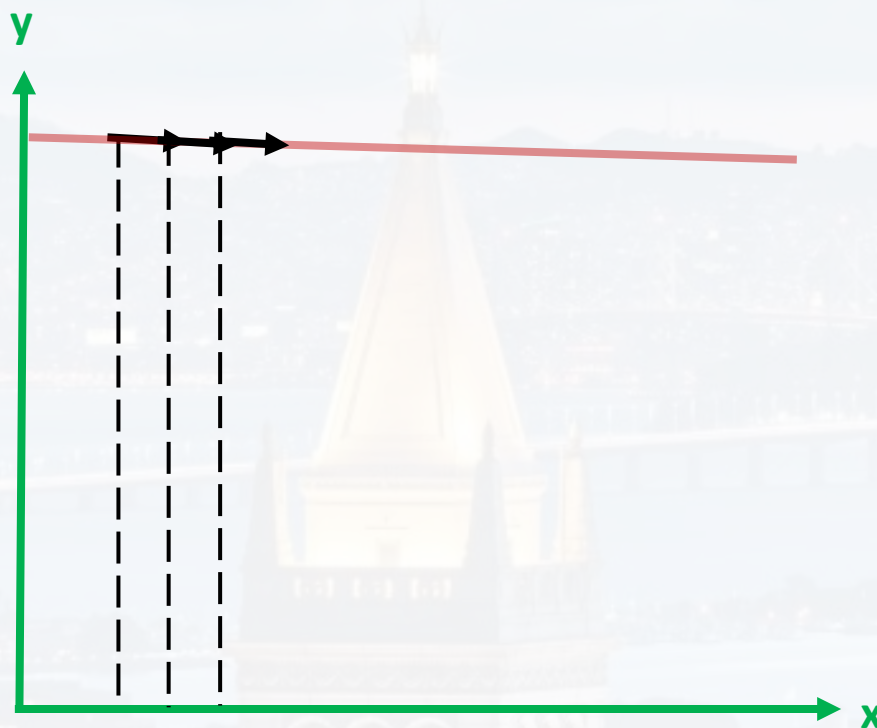
$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is

... and so on...

→ smaller ϵ ?

Takes too long!





$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

learning rate as function of t:

$$\epsilon > 0$$

called *learning rate*

$$\epsilon(t) = \frac{\epsilon_0}{1 + \kappa t} \quad \text{decay rate } \kappa$$

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is





$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

learning rate as function of t:

$$\epsilon > 0$$

called *learning rate*

$$\epsilon(t) = \frac{\epsilon_0}{1 + \kappa t} \quad \text{decay rate } \kappa$$

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is

can also be a stepwise function (learning rate schedule)



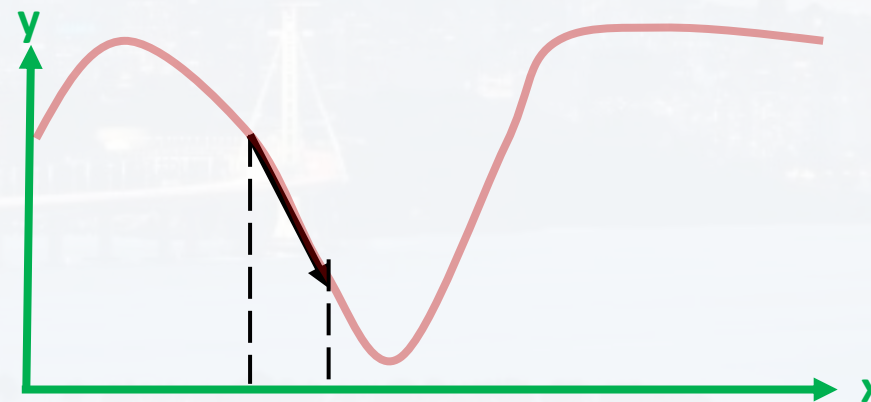
learning rate as function of t:

$$\epsilon(t) = \frac{\epsilon_0}{1 + \kappa t} \quad \text{decay rate } \kappa$$

$$\Delta x = -\epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large the leap Δx is

can also be a stepwise function (learning rate schedule)



$$\epsilon \rightarrow \frac{\epsilon}{\sqrt{\text{grad}(y)_x}}$$

adaptive gradient, aka **AdaGrad**



Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



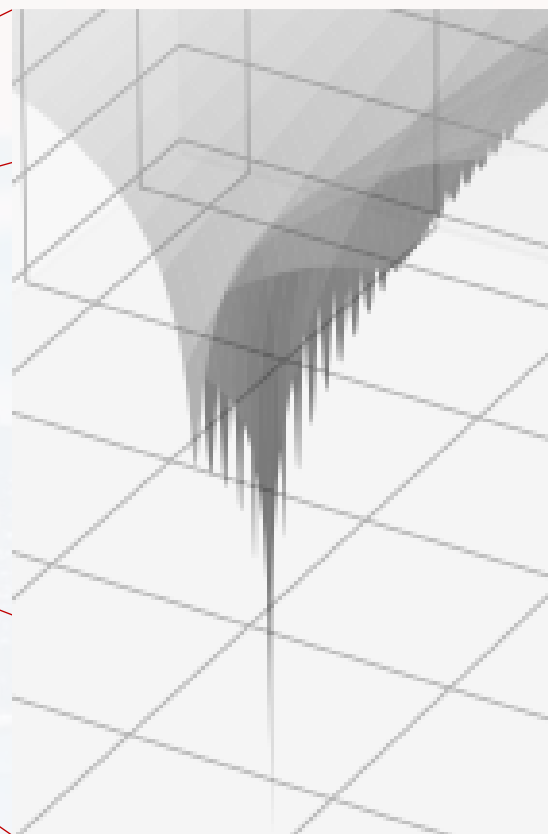
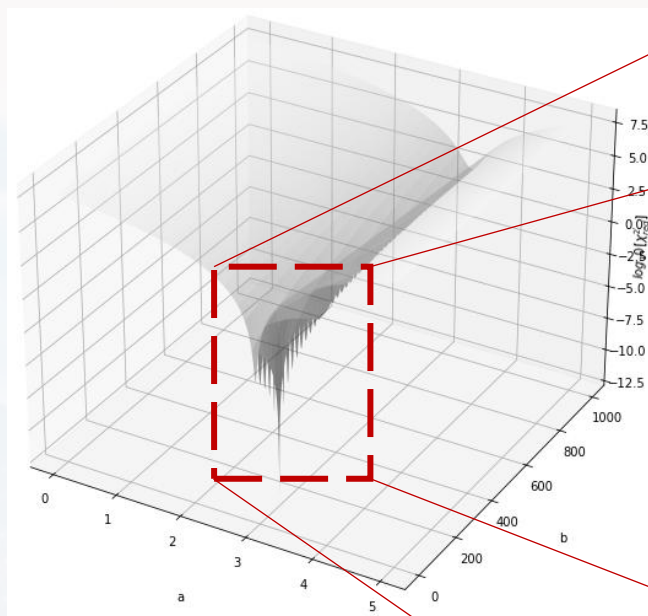
source: SKRyanrr

Outline

Lagrangian Multiplier

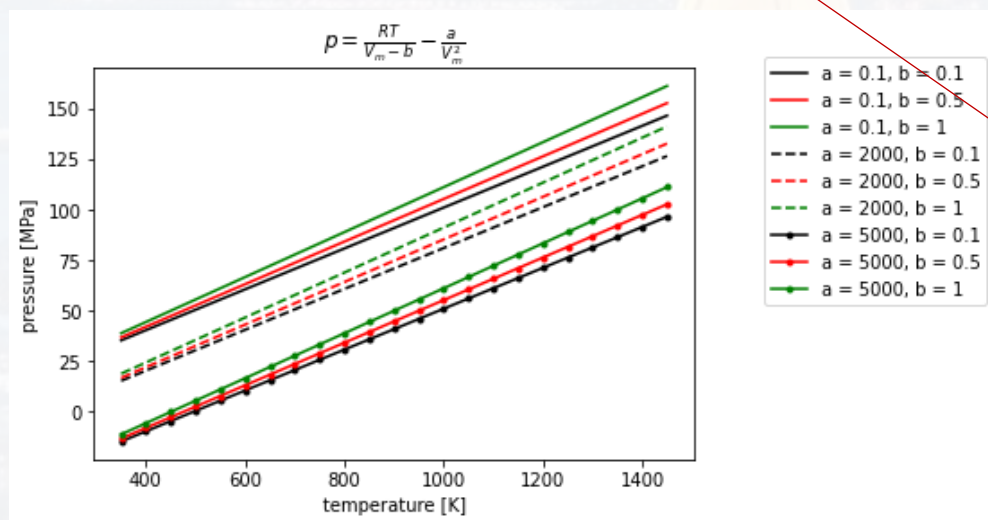
Gradient Descent Again

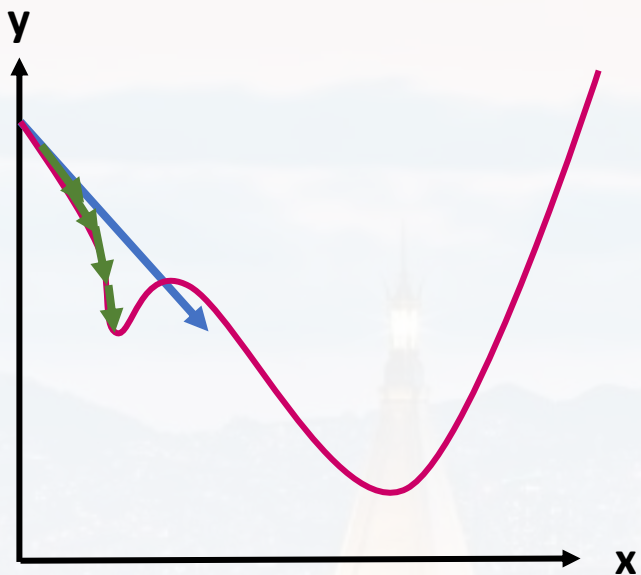
- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning



even with AdaGrad and learning rate schedule
→ would get stuck in local minimum

need to roll over → **momentum**





taking the **average** of N previous gradients

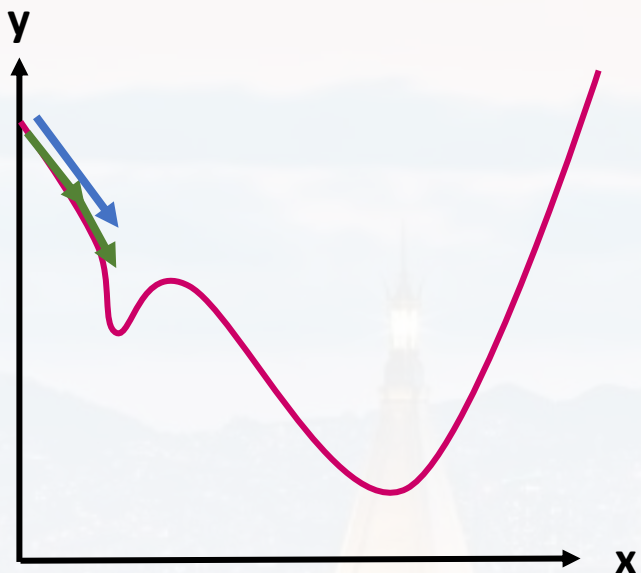
$$\langle grad(y)_{x(t)} \rangle = \frac{1}{N} [grad(y)_{x(t-1)} + grad(y)_{x(t-2)} + \dots + grad(y)_{x(t-N)}]$$

but we want more recent gradients to contribute more than older gradients

→ **weighted average** with weighting factor μ_k

$$\langle grad(y)_{x(t)} \rangle = \sum_{k=t-N}^{t-1} \mu_k \cdot grad(y)_{x(k)}$$

Finding a clever way to adjust μ_k during every iteration t

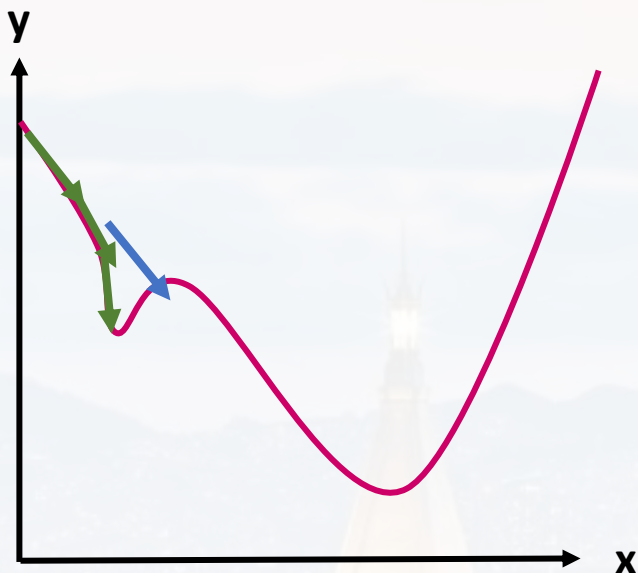


weighted average with weighting factor μ_k

Finding a clever way to adjust μ_k during every iteration t

$$\langle grad(y)_{x(0)} \rangle = grad(y)_{x(0)} \quad \mu_0 = (0,1)$$

$$\langle grad(y)_{x(1)} \rangle = grad(y)_{x(1)} + \mu_0 \cdot grad(y)_{x(0)}$$



weighted average with weighting factor μ_k

Finding a clever way to adjust μ_k during every iteration t

$$\langle grad(y)_{x(0)} \rangle = grad(y)_{x(0)} \quad \mu_0 = (0,1)$$

$$\langle grad(y)_{x(1)} \rangle = grad(y)_{x(1)} + \mu_0 \cdot grad(y)_{x(0)}$$

$$\langle grad(y)_{x(2)} \rangle = grad(y)_{x(2)} + \mu_0 [grad(y)_{x(1)} + \mu_0 grad(y)_{x(0)}]$$

$$\mu_{k=2} = \mu_0 \mu_0 = \mu_0^2$$

$$\langle grad(y)_{x(3)} \rangle = grad(y)_{x(3)} + \mu_0 [grad(y)_{x(2)} + \mu_0 [grad(y)_{x(1)} + \mu_0 \cdot grad(y)_{x(0)}]]$$

... and so on...

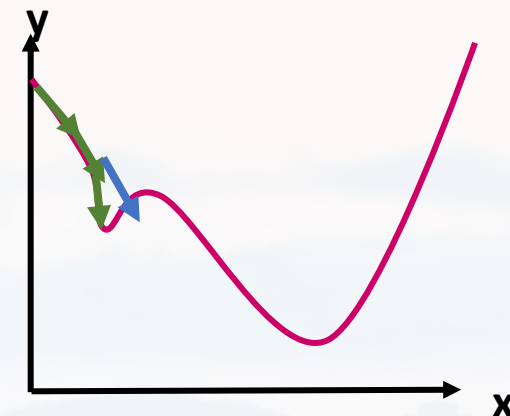


weighted average with weighting factor μ_k

$\mu_0 = (0,1)$ called "momentum"

$$\langle \text{grad}(\mathbf{y})_{x(3)} \rangle = \text{grad}(\mathbf{y})_{x(3)} +$$

$$\mu_0 \left[\text{grad}(\mathbf{y})_{x(2)} + \mu_0 \left[\text{grad}(\mathbf{y})_{x(1)} + \mu_0 \cdot \text{grad}(\mathbf{y})_{x(0)} \right] \right] \quad \dots \text{and so on...}$$



```
class Optimizer:
```

```
def __init__(self, learning_rate = 0.1, decay = 0, momentum = 0):  
    self.learning_rate      = learning_rate  
    self.decay              = decay  
    self.current_learning_rate = learning_rate  
    self.iterations         = 0  
    self.momentum           = momentum
```



Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



source: SKRyanrr

Outline

Lagrangian Multiplier

Gradient Descent Again

- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

sometimes we have some **prior knowledge** about the **independent variables**

recall: linear regression

finding best β by

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 \right\}$$

now:

constrain: **encourages sparsity of β**

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 + \lambda \|\beta\|^1 \right\}$$

the Lagrangian
 $L(X, Y, \lambda)$

λ Lagrangian Multiplier

called **L1 regularization**, or LASSO

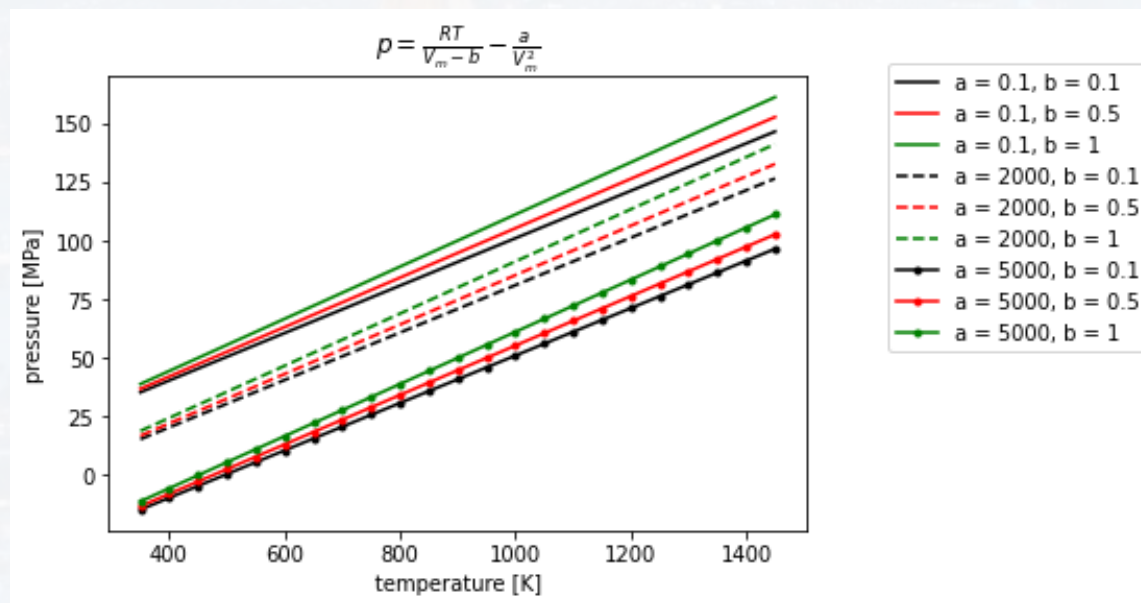


Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

sometimes we have some **prior knowledge** about the **independent variables**

L1 regularization



We often have even hard constraints based on the laws of physics!



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

sometimes we have some **prior knowledge** about the **independent variables**

recall: linear regression

finding best β by

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 \right\}$$

now:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T Y \longrightarrow$$

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 + \lambda \|\beta\|^2 \right\}$$

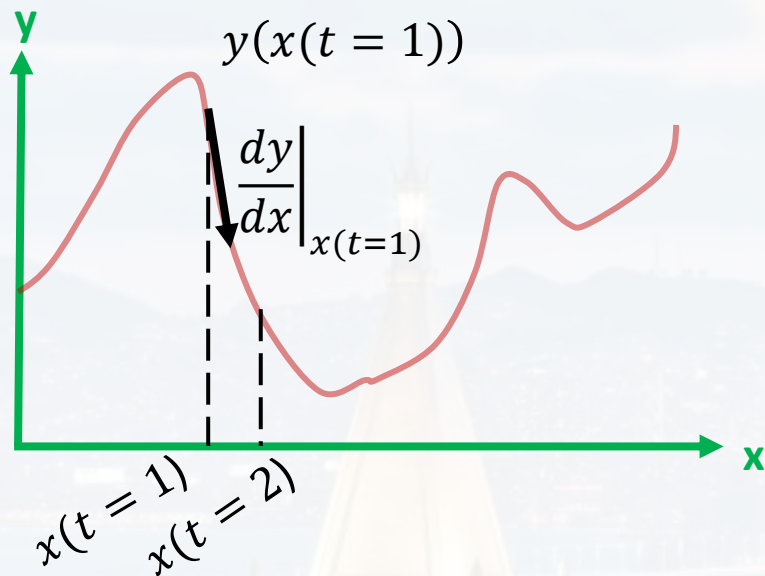
the Lagrangian
 $L(X, Y, \lambda)$

λ Lagrangian Multiplier

called **L2 regularization**, or RIDGE penalizes large β



L1 and L2 regularization



$$x(t=2) = x(t=1) - \varepsilon \frac{d[y + \lambda_1 \|x\|^1 + \lambda_2 \|x\|^2]}{dx} \Big|_{x(t=1)}$$

$$x(t=2) = x(t=1) - \varepsilon \frac{dy}{dx} \Big|_{x(t=1)}$$

$$- \varepsilon \frac{\lambda_1 d\|x\|^1}{dx} \Big|_{x(t=1)} - \varepsilon \frac{\lambda_2 d\|x\|^2}{dx} \Big|_{x(t=1)}$$

- gradient descent does not stop if values for x are too large and prefers sparsity
- note: the derivative of $\|x\|^1$ returns the sign (i. e. direction)
- usually $\lambda \ll \|x\|^n$
- will be important for ANNs later



Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism



source: SKRyanrr

Outline

Lagrangian Multiplier

Gradient Descent Again

- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning



Vanilla Gradient Descent → **S**tochastic **G**radient **D**escent



Learning Rate Schedule, L1, L2



Momentum



$$\epsilon \rightarrow \frac{\epsilon}{\sqrt{\text{grad}(y)_x}}$$

adaptive gradient, aka **AdaGrad**



Multiplying a decay factor to the sum of gradient squared (similar to momentum) in,

$$\epsilon \rightarrow \frac{\epsilon}{\sqrt{\text{moving_avg}(\text{grad}(y)_x)}}$$

aka **Root Mean Square Propagation** **RMSProp**

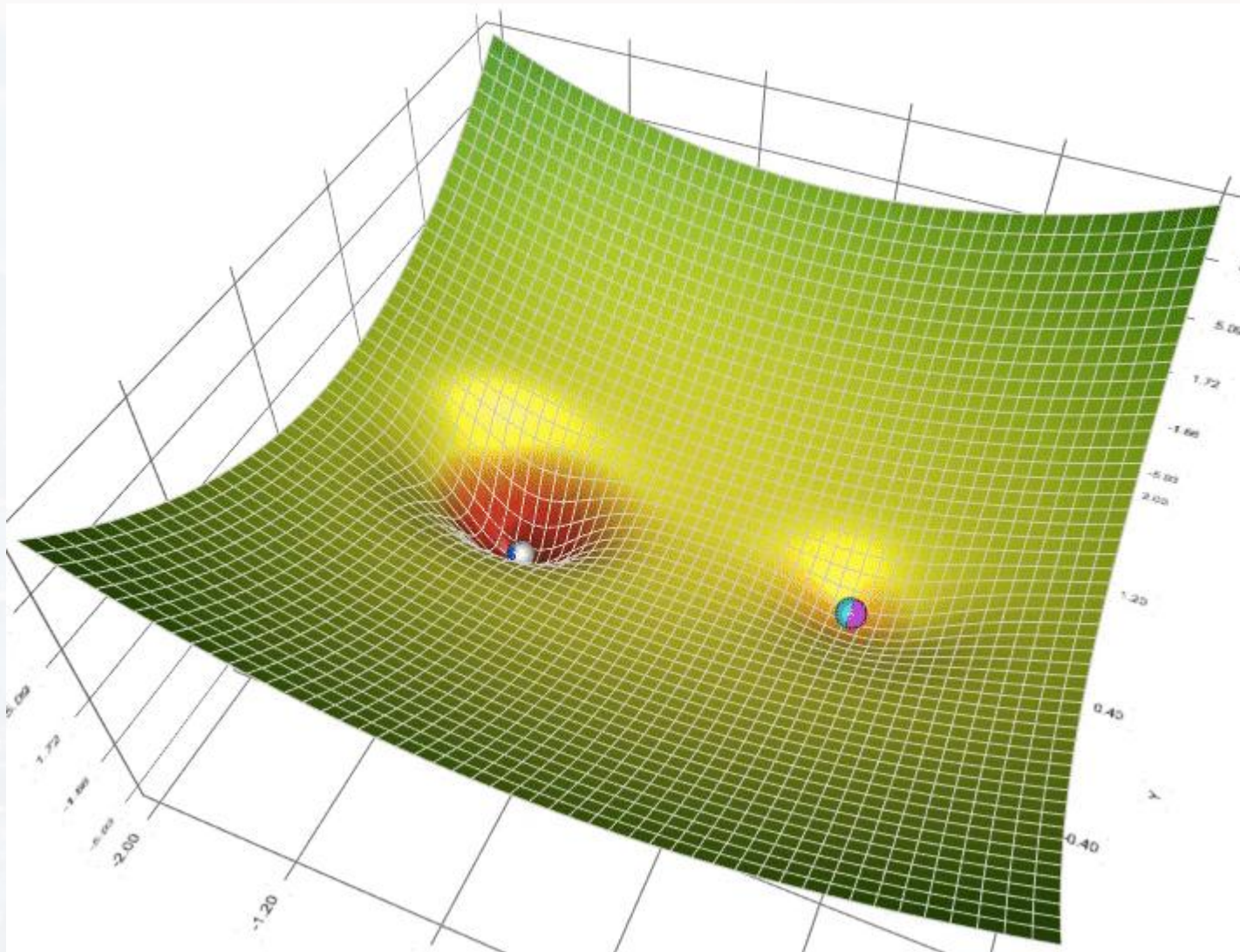
different scaling for all different directions

all combined:
Adaptive Moment Estimation
aka **Adam**



Lili Jiang

[TowardsDataScience](#)

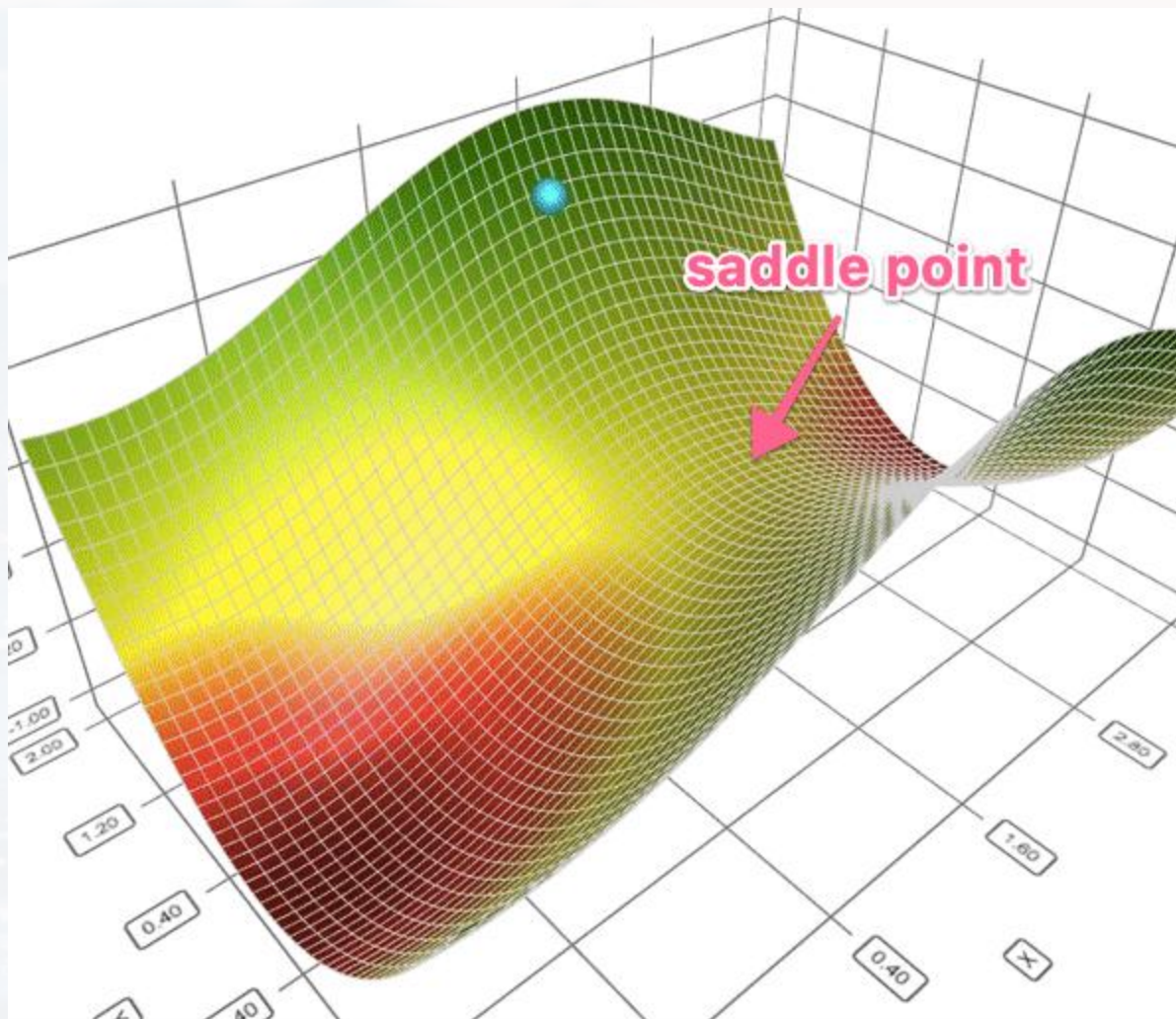


gradient descent (cyan),
momentum (magenta),
AdaGrad (white),
RMSProp (green),
Adam (blue)



Lili Jiang

[TowardsDataScience](#)



gradient descent (cyan),
momentum (magenta),
AdaGrad (white),
RMSProp (green),
Adam (blue)



Thank you very much for your attention!

Me on my way to learn Lagrangian formalism



Me after learning the Lagrangian formalism

